



Sistemas de Inteligencia Artificial
Métodos de Búsqueda
Desinformados



Recapitulando...



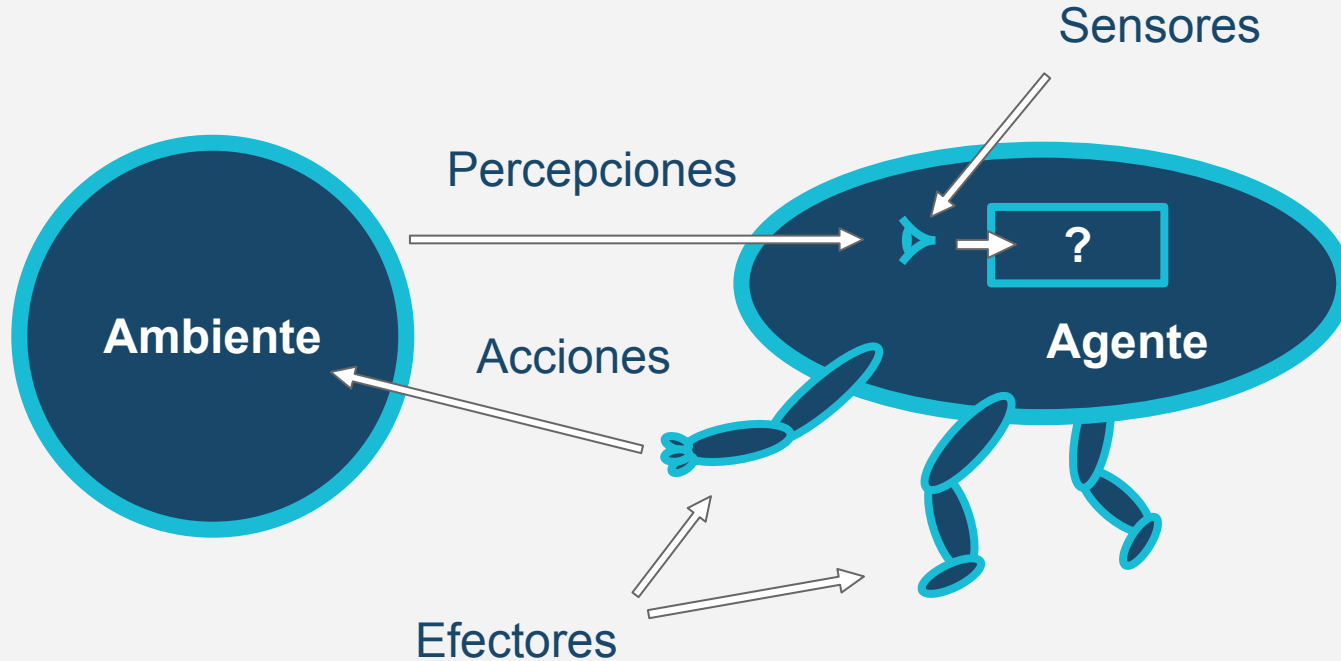
Sistemas de Inteligencia Artificial

Agentes y Ambientes





El Agente y su Ambiente





El Agente y su Ambiente



Sistemas de detección de fraude

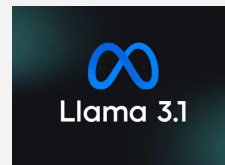


Sistemas de recomendación



ChatGPT

Gemini



Agentes de inteligencia artificial generativa





Agentes

Todo aquello que puede considerarse que **percibe** y **responde** o actúa en un **ambiente**, de forma **independiente** (autónomo).

Racionales vs **No Racionales**

Omniscientes vs **No Omniscientes**

Sin vs **Con** estado interno





Ambientes

Es un conjunto de componentes que conforman al **entorno** del problema.
Pueden contener uno o más agentes.

1. **Totalmente** vs **Parcialmente** Observable
2. **Determinístico** vs **Estocástico** (*vs Estratégico*)
3. **Episódico** vs **Secuencial**
4. **Estático** vs **Dinámico** (*vs Semidinámico*)
5. **Discreto** vs **Continuo**
6. **Conocido** vs **Desconocido**
7. **Individual** vs **Multiagente**
8. **Adversarial** vs **Colaborativo**





Sistemas de Inteligencia Artificial

Métodos de Búsqueda

Desinformados





Definiciones

- **Estado:** Conjunto de valores del ambiente en un momento dado.
- **Acción:** Una función que toma un estado y retorna otro estado, si es aplicable.
- **Espacio de estados:** Conjunto de estados que incluye todos los estados alcanzables desde el estado inicial.
- **Espacio de acciones:** Conjunto de acciones que incluye todas las acciones aplicables a por lo menos un estado en el espacio de estados.





Información que se utiliza para crear al agente que realiza un método de búsqueda.

- Se analizan los objetivos a lograr.
- Se formulan las acciones posibles y los estados (o su arquitectura).
- Se elabora un algoritmo de búsqueda que:
 - Recibe un estado inicial (ambiente).
 - Ejecuta indefinidamente o hasta llegar a cierto(s) estado(s).





Problemas Bien Definidos

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

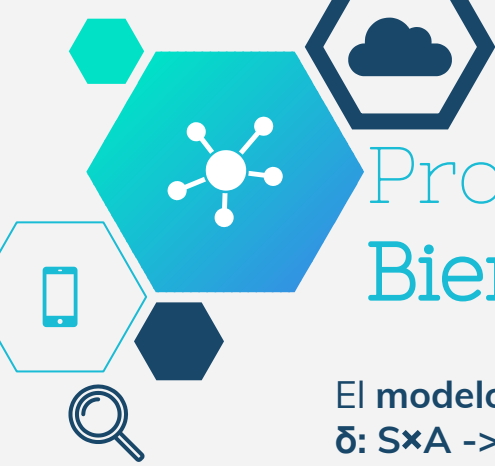
Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

Condición de
solución:

goal: $S \rightarrow \{0, 1\}$

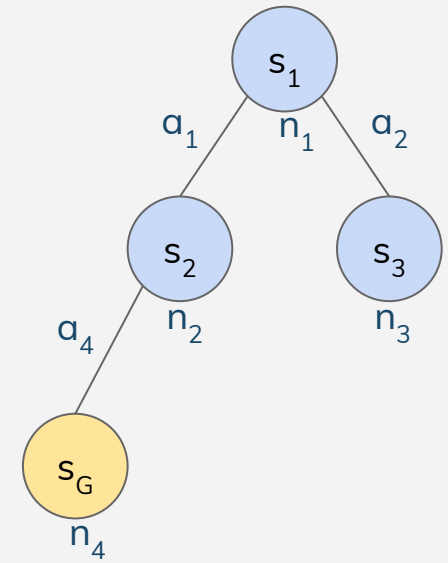




Problemas Bien Definidos

El modelo de transición es:
 $\delta: S \times A \rightarrow S$ o también $\delta(s_x, a_y) = s_z$

δ	a_1	a_2	a_3	a_4
s_1	s_2	s_3	$\emptyset$	$\emptyset$
s_2	s_1	$\emptyset$	$\emptyset$	s_G
s_3	$\emptyset$	$\emptyset$	$\emptyset$	s_1
s_G	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$



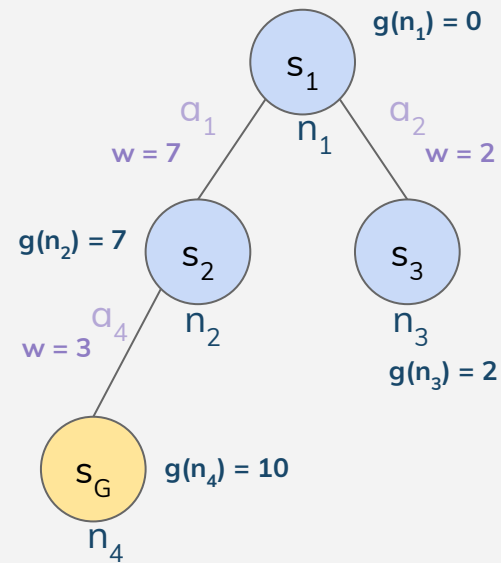


Problemas Bien Definidos

Existe un **costo** asociado a aplicar una acción sobre un estado:

$g: S \times A \rightarrow \mathbb{R}$ o también $g(s_x, a_y) = w$

δ	α_1	α_2	α_3	α_4		
s_1	s_2	g	α_1	α_2	α_3	α_4
s_2	s_1	s_1	7	2	$\emptyset$	$\emptyset$
s_3	$\emptyset$	s_2	7	$\emptyset$	$\emptyset$	3
s_G	$\emptyset$	s_3	$\emptyset$	$\emptyset$	$\emptyset$	2
		s_G	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$





Problema Ejemplo

Información que se utiliza para crear al agente que realiza un método de búsqueda.

- Se analizan los objetivos a lograr.
- Se formulan las acciones posibles y los estados (o su arquitectura).
- Se elabora un algoritmo de búsqueda que:
 - Recibe un estado inicial (ambiente).
 - Ejecuta indefinidamente o hasta llegar a cierto(s) estado(s).



Pokémon Emerald Version

Objetivo: ¿?

- Conseguir todos los items
- Ganar todos los concursos
- Ganar el frente de batalla (obtener todos los símbolos)
- **Ganar las 8 medallas**
- **Ganar la liga Pokémon**





Problema Ejemplo



Laberinto

Información que se utiliza para crear al agente que realiza un método de búsqueda.

- Se analizan los objetivos a lograr.
- Se formulan las acciones posibles y los estados (o su arquitectura).
- Se elabora un algoritmo de búsqueda que:
 - Recibe un estado inicial (ambiente).
 - Ejecuta indefinidamente o hasta llegar a cierto(s) estado(s).

Objetivo:

- Llegar a la bolsita de cereales

Estado inicial:

- Estoy en el tazón de cereales





Definiciones

Ejemplo

- **Estado:** Conjunto de valores del ambiente en un momento dado.
- **Acción:** Una función que toma un estado y retorna otro estado, si es aplicable.
- **Espacio de estados:** Conjunto de estados que incluye todos los estados alcanzables desde el estado inicial.
- **Espacio de acciones:** Conjunto de acciones que incluye todas las acciones aplicables a por lo menos un estado en el espacio de estados.



Laberinto

- **Espacio de acciones**

¿Qué definimos como **acción** en este contexto?

- **Espacio de estados**

¿Cómo definimos un **estado** en este contexto?

definir // en este contexto





Definiciones

Ejemplo

- **Estado:** Conjunto de valores del ambiente en un momento dado.
- **Acción:** Una función que toma un estado y retorna otro estado, si es aplicable.
- **Espacio de estados:** Conjunto de estados que incluye todos los estados alcanzables desde el estado inicial.
- **Espacio de acciones:** Conjunto de acciones que incluye todas las acciones aplicables a por lo menos un estado en el espacio de estados.



Laberinto

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Espacio de estados**

$S = \{s \mid s \text{ contenga P, L y O}\}$

- P: posición del trazo
- L: límites del laberinto
- O: posiciones bloqueadas (paredes)





Problema Ejemplo



Laberinto

Información que se utiliza para crear al agente que realiza un método de búsqueda.

- Se analizan los objetivos a lograr.
- Se formulan las acciones posibles y los estados (o su arquitectura).
- Se elabora un algoritmo de búsqueda que:
 - Recibe un estado inicial (ambiente).
 - Ejecuta indefinidamente o hasta llegar a cierto(s) estado(s).

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Espacio de estados**

$S = \{s \mid s \text{ contenga P, L y O}\}$

- P: posición del trazo
- L: límites del laberinto
- O: posiciones bloqueadas (paredes)





Problema Ejemplo

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.



Laberinto

- **Estado inicial**

Estoy en el tazón de cereales

- P = tazón
- L = franjas blancas más externas
- O = todo aquello pintado de blanco dentro de L

- **Espacio de estados**

$S = \{ s \mid s \text{ contenga } P, L \text{ y } O \}$

- P : posición del trazo
- L : límites del laberinto
- O : posiciones bloqueadas (paredes)





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Espacio de estados**

$S = \{s \mid s \text{ contenga } P, L \text{ y } O\}$

- P : posición del trazo
- L : límites del laberinto
- O : posiciones bloqueadas (paredes)





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Espacio de estados**

$S = \{s \mid s \text{ contenga } P, L \text{ y } O\}$

- P : posición del trazo
- L : límites del laberinto
- O : posiciones bloqueadas (paredes)





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Espacio de estados**

$S = \{s \mid s \text{ contenga } P, L \text{ y } O\}$

- P : posición del trazo
- L : límites del laberinto
- O : posiciones bloqueadas (paredes)





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Espacio de estados**

$S = \{s \mid s \text{ contenga } P, L \text{ y } O\}$

- P : posición del trazo
- L : límites del laberinto
- O : posiciones bloqueadas (paredes)





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Espacio de estados**

$S = \{s \mid s \text{ contenga } P, L \text{ y } O\}$

- P : posición del trazo
- L : límites del laberinto
- O : posiciones bloqueadas (paredes)





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Costo**

- Longitud del trazo





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Espacio de acciones**

$A = \{a \mid a \text{ sea un movimiento del trazo que no cruce lo blanco}\}$

- Muevo hacia abajo

- **Costo**

- Longitud del trazo





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Objetivo:**
 - Llegar a la bolsita de cereales
- **Espacio de estados**

$S = \{ s \mid s \text{ contenga } P, L \text{ y } O \}$

 - P : posición del trazo
 - L : límites del laberinto
 - O : posiciones bloqueadas (paredes)
- **Condición de solución**
 - $\text{goal}(s) == 1 \Leftrightarrow P_s == \text{bolsita de cereales}$





Representación de Problema

- Definición de arquitectura de estados...
 - Estratégicamente, entendiendo los operadores entre ellos.
 - De forma tal que representen todo el espacio de estados.
 - Entendiendo el costo computacional de dicha representación y de sus operadores.
 - Buscando simetría de estados.
 - El conjunto de acciones es más reducido (manteniendo utilidad).



Representación de Problema



Laberinto

- **Difícil representar computacionalmente**
- **Acciones:** Continuo de posibilidades para el trazo (¡INFINITAS ACCIONES!)
 - Muevo hacia abajo 0.123° respecto de la vertical a la izquierda (-0.123°)
 - ¿Cuánto muevo? ¿2cm? ¿2 pixeles?
- **Estados:** Posición en un continuo (¡INFINITOS ESTADOS!)

Puede resultar costoso computacionalmente detectar si en x posición hay un obstáculo

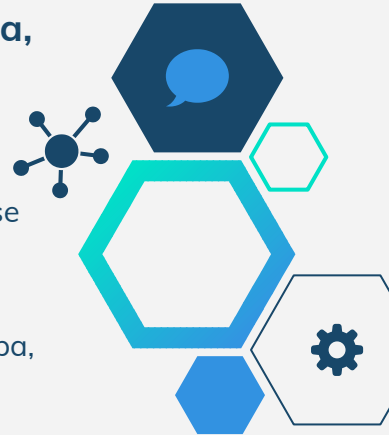


Representación de Problema



Laberinto

- **¡Podemos discretizar el problema!**
 - Si hay blanco en el cuadrado, esta bloqueado
- **$A = \{ \text{arriba, abajo, izquierda, derecha} \}$**
- **$S = \{ s \mid s = (P, L) \}$ donde**
 - P es el cuadradito en el que se encuentra el jugador
 - L tiene la disponibilidad de cada cuadradito (matriz, mapa, etc.)





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Objetivo:** Llegar a la bolsita de cereales
- **Estado inicial:** tazón de cereales
- $A = \{ \text{arriba, abajo, izquierda, derecha} \}$
- $S = \{ s \mid s = (P, L) \}$
- $\delta(s, a)$: mueve P según a
- $\forall a \in A, g(a) = 1$
- $\text{goal}(s) == 1 \Leftrightarrow P_s == \text{bolsita de cereales}$





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Objetivo:** Llegar a la bolsita de cereales
- **Estado inicial:** tazón de cereales
- $A = \{ \text{arriba, abajo, izquierda, derecha} \}$
- $S = \{ s \mid s = (P, L) \}$
- $\delta(s, a)$: mueve P según a
- $\forall a \in A, g(a) = 1$
- $\text{goal}(s) == 1 \Leftrightarrow P_s == \text{bolsita de cereales}$





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Objetivo:** Llegar a la bolsita de cereales
- **Estado inicial:** tazón de cereales
- $A = \{ \text{arriba, abajo, izquierda, derecha} \}$
- $S = \{ s \mid s = (P, L) \}$
- $\delta(s, a)$: mueve P según a
- $\forall a \in A, g(a) = 1$
- $\text{goal}(s) == 1 \Leftrightarrow P_s == \text{bolsita de cereales}$





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Objetivo:** Llegar a la bolsita de cereales
- **Estado inicial:** tazón de cereales
- $A = \{ \text{arriba, abajo, izquierda, derecha} \}$
- $S = \{ s \mid s = (P, L) \}$
- $\delta(s, a)$: mueve P según a
- $\forall a \in A, g(a) = 1$
- $\text{goal}(s) == 1 \Leftrightarrow P_s == \text{bolsita de cereales}$





Problema Ejemplo



Laberinto

Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.

Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .

Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).

Función de costo: Determina el costo para el modelo de transición.

Condición de solución: Función que nos indica si un estado S es solución.

- **Objetivo:** Llegar a la bolsita de cereales
- **Estado inicial:** tazón de cereales
- $A = \{ \text{arriba, abajo, izquierda, derecha} \}$
- $S = \{ s \mid s = (P, L) \}$
- $\delta(s, a)$: mueve P según a
- $\forall a \in A, g(a) = 1$
- $\text{goal}(s) == 1 \Leftrightarrow P_s == \text{bolsita de cereales}$





Representación de Problema

8		6
5	4	7
2	3	1



2	5	8
3	4	
1	7	6

8 puzzle



Representación de Problema

2	5	8
3	4	
1	7	6



2	5	8
3	4	
1	7	6

8 puzzle

Los estados podrían definirse como los números adyacentes a un número





Definiendo Acciones ...

- Teniendo en cuenta las suposiciones de la descripción informal del problema.
- Eligiendo entre acciones particulares o meta-acciones*.
 - **Meta-acciones** es una forma de definir genéricamente un conjunto de acciones.
- Decidiendo cuanto debe ser resuelto en el precálculo de acciones y cuánto debe ser resuelto por el método de búsqueda.





Definiendo Acciones ...



Pokémon Emerald Version

Mirar en una dirección puede resultar una acción sin utilidad



Definiendo Acciones ...



Pokémon Emerald Version

3 acciones, una única acción: Moverse a la izquierda

- Mirar a la izquierda
- Moverse a la izquierda
- Mirar al frente

Defino una meta acción de moverme a la izquierda





Definiendo Acciones ...



4 acciones:
arriba, abajo,
izquierda,
derecha





Definiendo Acciones ...



¿Podría definir las meta acciones de movimiento diagonal?





Definiendo Acciones ...



Pokémon Emerald Version

Sí pero OJO. Si la definimos cómo arriba-izquierda o izquierda-arriba CAMBIA. Tener en cuenta las implicancias.





Eficiencia

Forma de medir la eficiencia de una búsqueda:

- Encuentra solución.
- Optimiza el costo de la solución.
 - Mínimo
 - Máximo
 - Promedio
- Optimiza la cantidad de acciones a tomar.
- Optimiza el costo computacional de encontrar una solución.
 - Tiempo
 - Memoria
 - ...
- Combinación
- Otros, según lo requiera el problema





Direcciones de búsqueda

Forward / “data-driven”

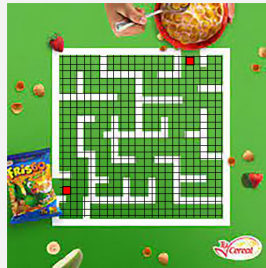
La raíz es el estado inicial.

Backward / “goal-driven”

La raíz es la solución (o una de las soluciones).



En el caso del laberinto, ¿cómo sería una forma de resolverlo “data-driven”? ¿y una “goal-driven”?



Laberinto

Podemos comenzar:

- desde el **tazón** (data-driven)
- o la **bolsita** (goal-driven)





Estrategias de búsqueda

Completa

Encuentra una solución (de existir y ser alcanzable).

Óptima

Encuentra la solución alcanzable con menor costo.

Complejidad computacional

- Temporal
- Espacial





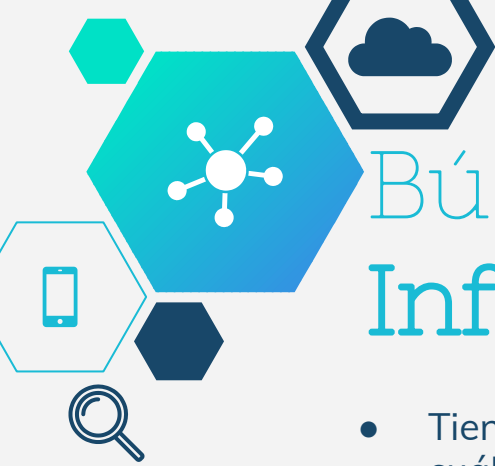
Búsquedas Desinformadas

- No tienen más información de los estados que los que otorga la definición del problema.
- Sólo se distinguen estados “goal” de los que no lo son.
- Poco eficientes en problemas donde podemos realizar estimaciones.

Algunos de ellos son:

- BFS
- DFS
- IDDFS
- Costo Uniforme





Búsquedas Informadas

- Tienen más información para comparar entre dos estados cuál es “mejor”.
- Pueden estimar cuánto les falta para solucionar el problema. La estimación a la solución desde un estado se llama **HEURÍSTICA**.
- Pueden estimar cuál es el costo del problema.
- En problemas complejos, suelen ser más eficientes que los métodos no informados.

Algunos de ellos son:

- Greedy
- A*





Representación de Problema

De esta forma se genera un **árbol** donde:

- La raíz representa al estado inicial
- Cada nodo representa (contiene) un estado
 - Los nodos **hoja** son estados sin acciones aplicables o estados que no fueron **expandidos**.



Un estado es **expandido** cuando dentro del método de búsqueda se le han aplicado todas las acciones aplicables.

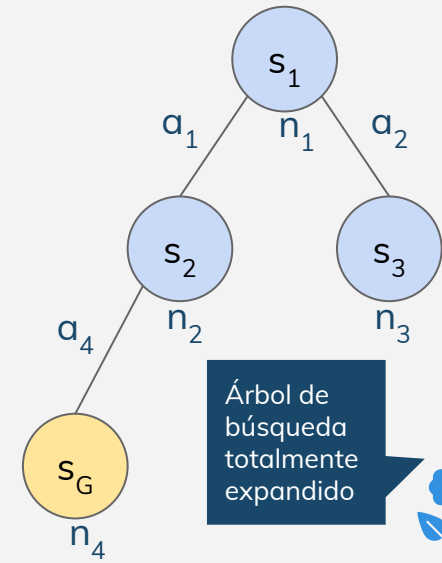




Representación de Problema

De esta forma se genera un **árbol** donde:

- La raíz representa al estado inicial
- Cada nodo representa (contiene) un estado
 - Los nodos **hoja** son estados sin acciones aplicables o estados que no fueron **expandidos**.

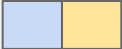


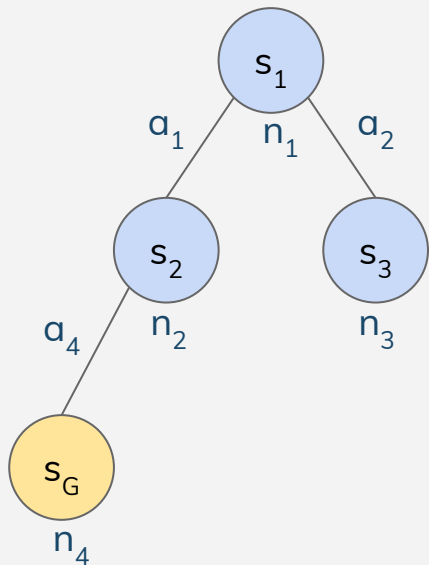
Un estado es **expandido** cuando dentro del método de búsqueda se le han aplicado todas las acciones aplicables.



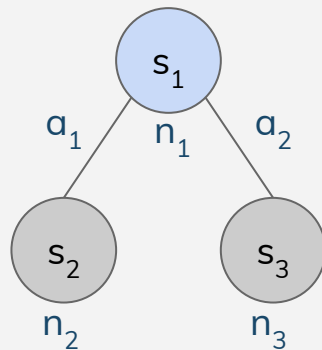


Representación de Problema

 Nodos no explorados
 Nodos explorados



Árbol de búsqueda totalmente expandido

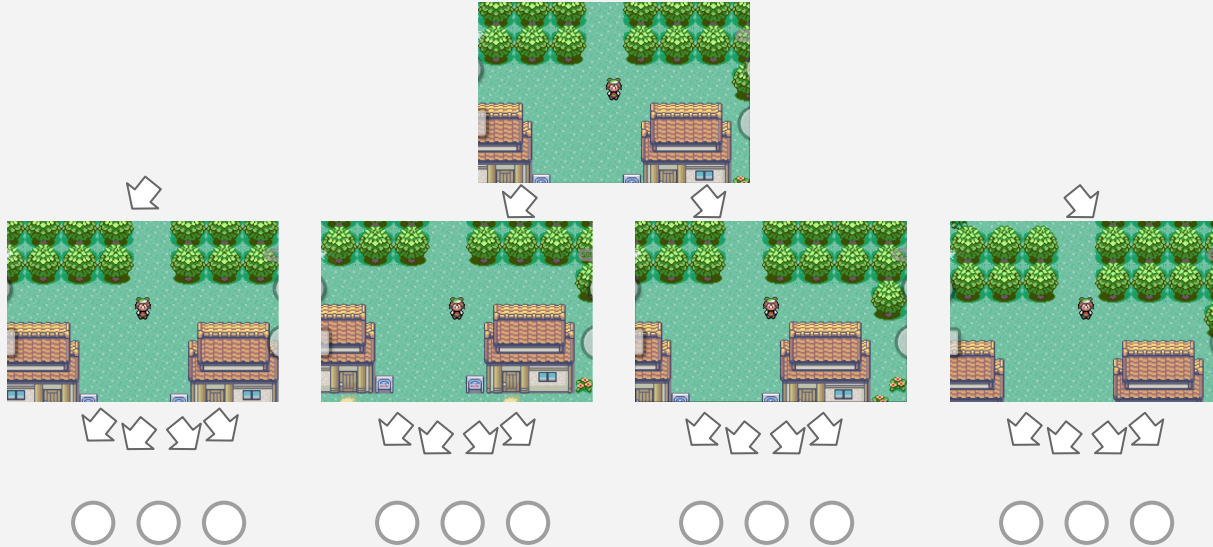


¿Qué nodos están expandidos?





Representación de Problema





Nodo $\neq$ Estado

No confundir NODO con ESTADO!

Un nodo contiene un estado. De esta forma, un estado puede estar representado en diferentes nodos.

Siendo **$g(n)$** el costo acumulado del nodo (la suma del arco desde n_0 hasta n), podemos encontrar dos nodos n_1 y n_2 , que pueden contener al mismo estado, de forma tal que:

$$g(n_1) \neq g(n_2)$$



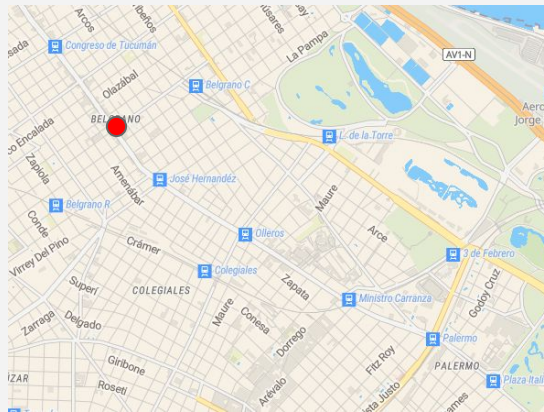
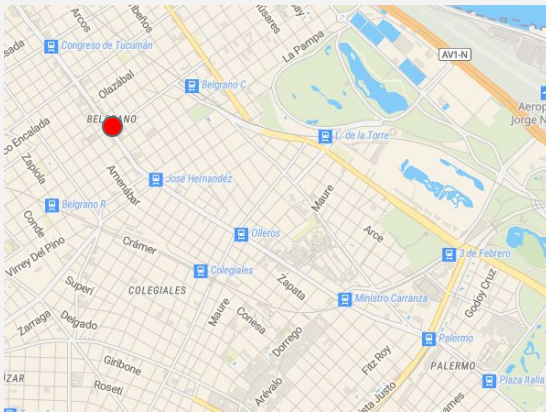
En rigor, n_1 y n_2 tienen diferentes antecesores.





Nodo $\neq$ Estado

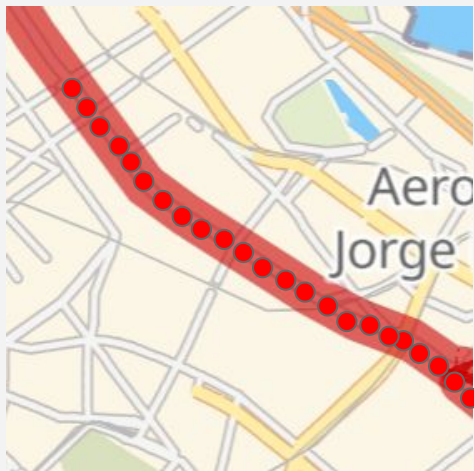
Imaginemos que queremos ir de **Plaza Italia** a **Congreso de Tucumán**. Analicemos estos estados:



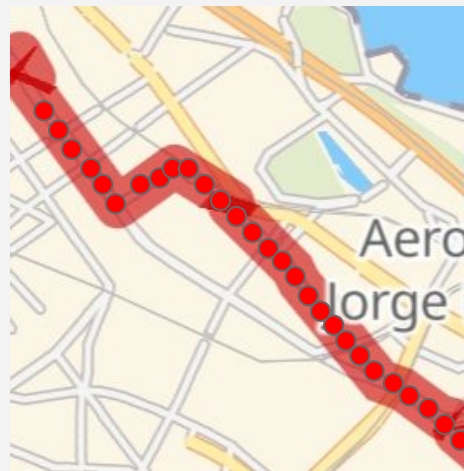


Nodo $\neq$ Estado

Imaginemos que queremos ir de **Plaza Italia** a **Congreso de Tucumán**. ¿Son contenidos por nodos iguales? **NO**



Línea 152 (Olivos)



Línea 29 (Pte Saavedra)

La sombra
representa el camino
solución desde mi
estado inicial al final.

El estado puede ser
la posición del
colectivo en un
momento dado.

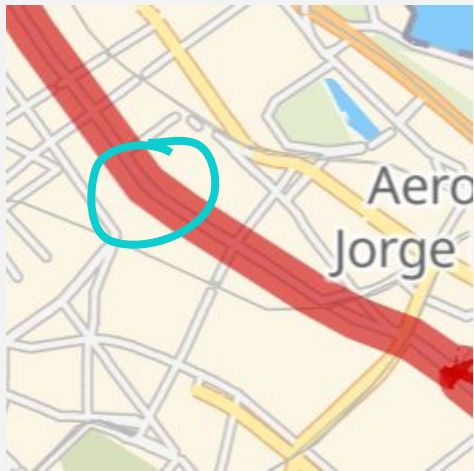




Nodo $\neq$ Estado

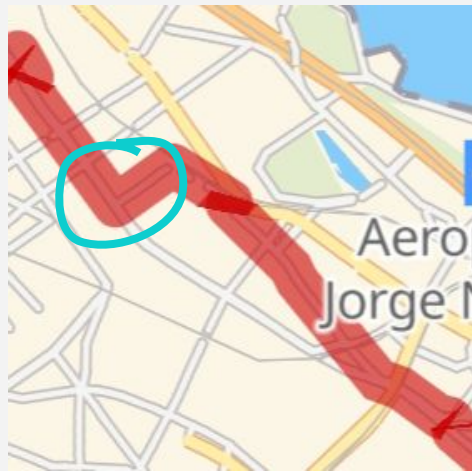
Imaginemos que queremos ir de **Plaza Italia** a **Congreso de Tucumán**. ¿Son contenidos por nodos iguales? **NO**

Por el
SURESTE



Línea 152 (Olivos)

Por el
NORESTE



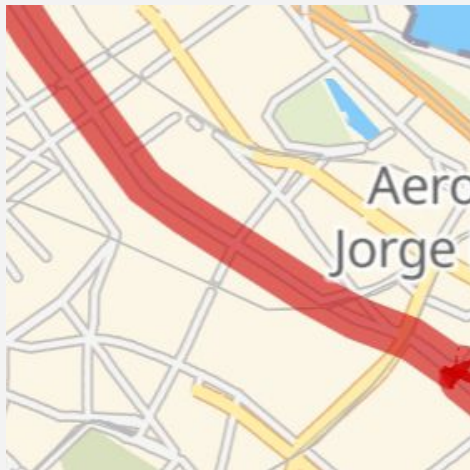
Línea 29 (Pte Saavedra)





Nodo $\neq$ Estado

Imaginemos que queremos ir de **Plaza Italia** a **Congreso de Tucumán**. ¿Son contenidos por nodos iguales? **NO**



Línea 152 (Olivos)



Línea 29 (Pte Saavedra)

Recorre **MÁS** distancia.

Tiene **MAYOR** costo.

NO es lo mismo.





Algoritmo de Búsqueda

Dados Árbol de búsqueda **Tr** ; Conjunto frontera **Fr** ; Conjunto de
nodos explorados **Exp** ...

1. Crear **Tr**, **Fr** (y **Exp**) inicialmente vacíos.
2. Insertar nodo inicial n_0 en **Tr** y **Fr**.
3. Mientras **Fr** no esté vacía... (6 en caso de estar vacía)
 - a. Extraer primer nodo de **Fr** => Nodo **n**
 - b. Si **n** es goal
 - i. Devolver la solución, formada por el arco entre la raíz n_0 y el nodo **n** en **Tr**. Termina **Algoritmo**.
 - c. Expandir el nodo **n**, generando los sucesores. Ingresar dichos sucesores en **Fr** y colocarlos en **Tr** como nodos sucesores de **n**. (Agregar **n** a **Exp**)





Algoritmo de Búsqueda

4. Reordenar **Fr** según un criterio dado (depende del método de búsqueda).
5. Ir al paso 3.

Si **Fr** está vacía en el paso 3...

6. No existe solución alcanzable.



También se puede tomar **Fr** como un conjunto ordenado, donde no haga falta reordenar.



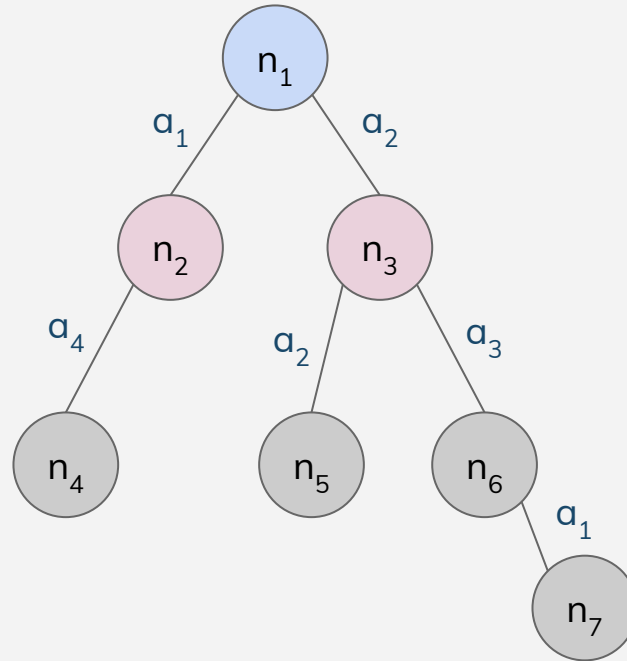


Algoritmo de Búsqueda

$Tr = \{n_1, n_2, n_3\}$

$Fr = \{n_2, n_3\}$

$Exp = \{n_1\}$

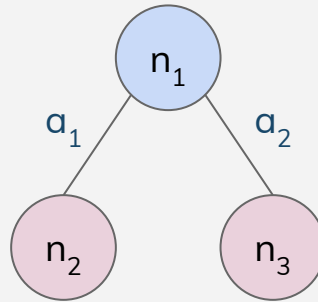


-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados





Algoritmo de Búsqueda



→ $Tr = \{ n_1, n_2, n_3 \}$

$Fr = \{ n_2, n_3 \}$

$Exp = \{ n_1 \}$

-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



En realidad, desconozco los estados futuros.
Al explorar un nodo, obtengo qué acciones le
puedo aplicar a su estado y, con ello, consigo los
siguientes estados



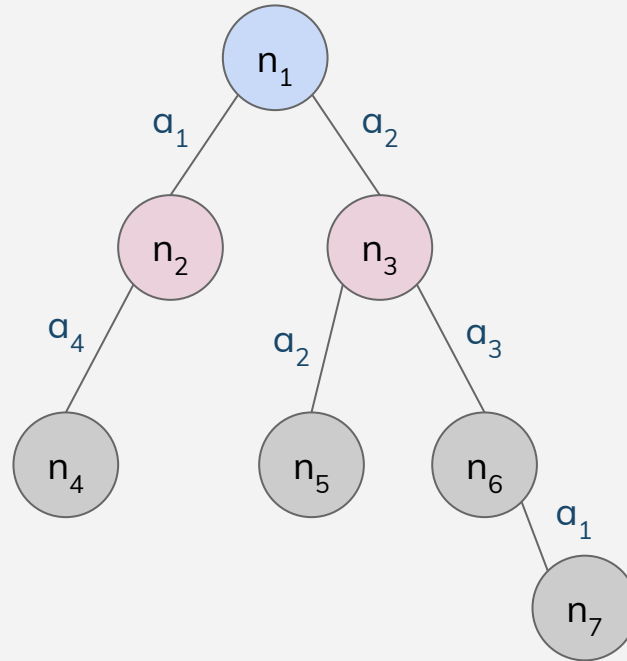


Algoritmo de Búsqueda

$Tr = \{n_1, n_2, n_3\}$

$Fr = \{n_2, n_3\}$

$Exp = \{n_1\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



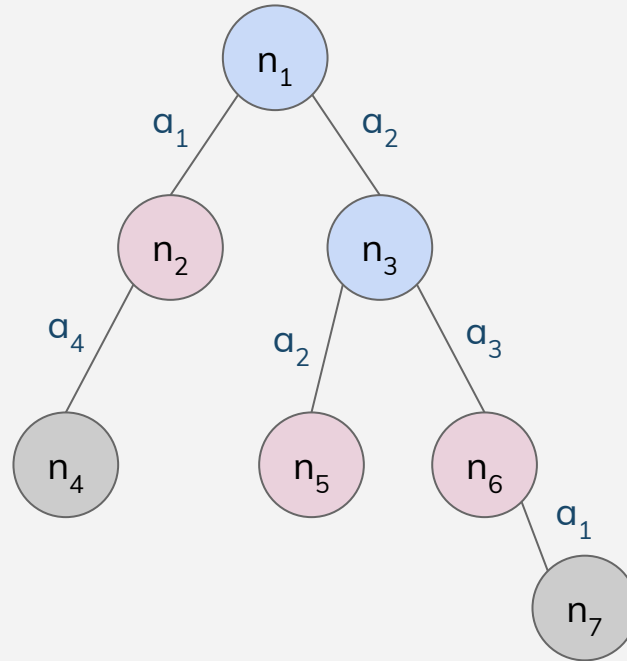


Algoritmo de Búsqueda

$Tr = \{n_1, n_2, n_3, n_5, n_6\}$

$Fr = \{n_2, n_5, n_6\}$

$Exp = \{n_1, n_3\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados

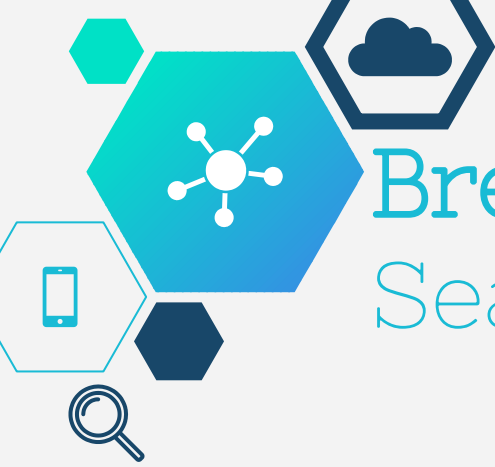




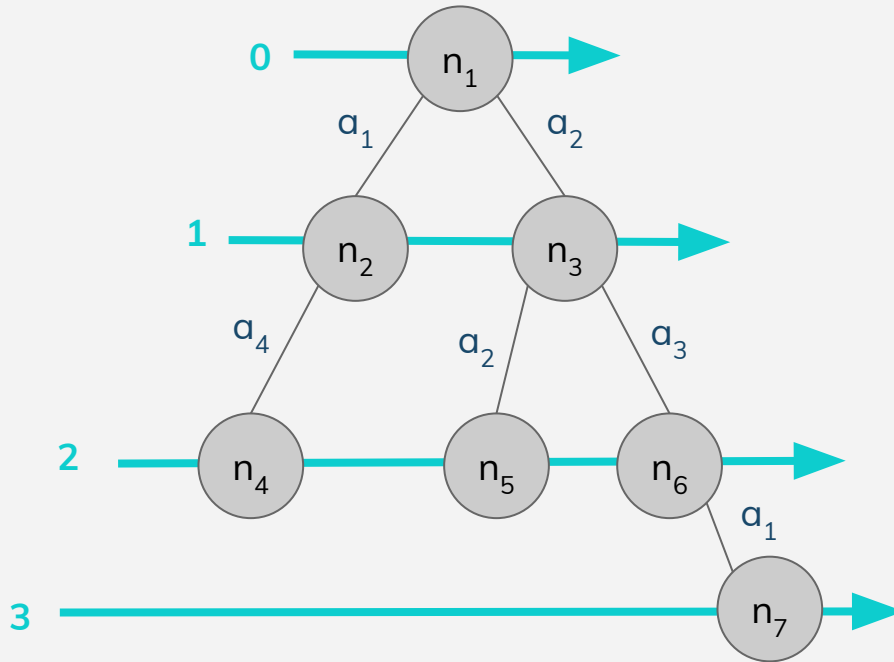
Breadth First Search

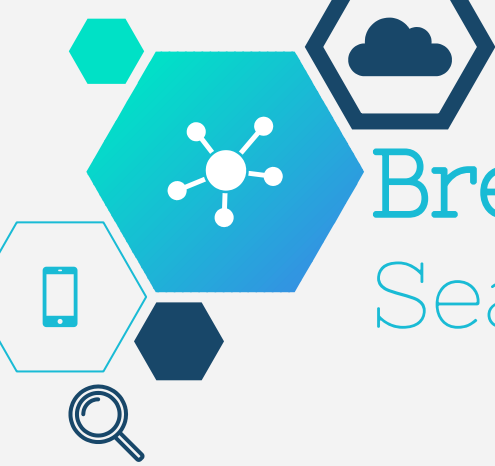
- Expande los nodos de menor profundidad primero.
- Es **completa** si el factor de ramificación es finito.
- Encuentra la solución con menor profundidad.
- Es **Óptima** cuando el problema es de costo uniforme (*).
 - (*) Las acciones tienen todas el mismo costo.





Breadth First Search



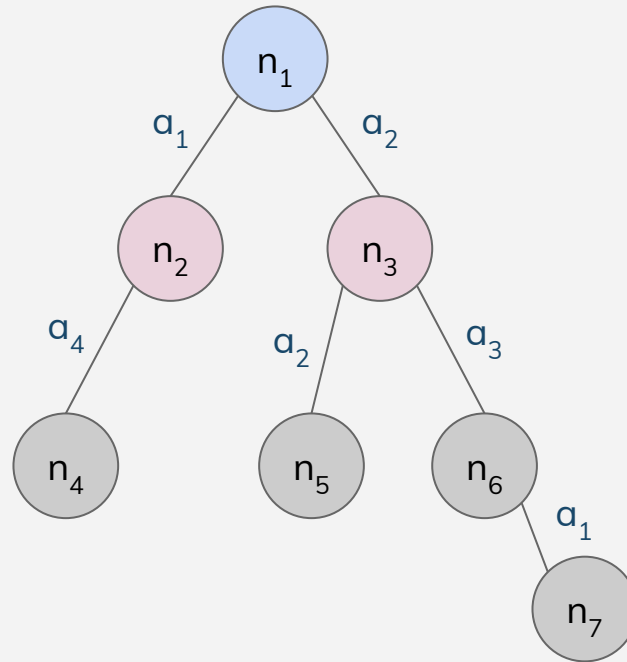


Breadth First Search

$Tr = \{n_1, n_2, n_3\}$

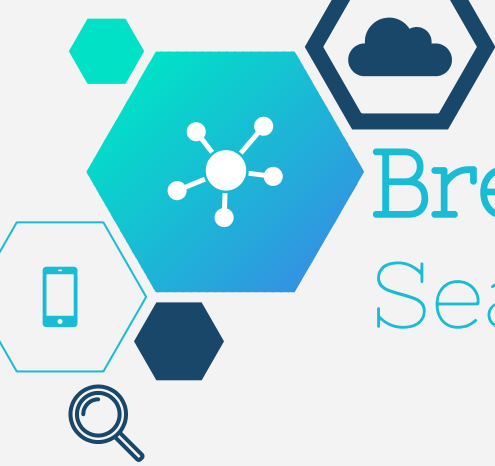
$Fr = \{n_2, n_3\}$

$Exp = \{n_1\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



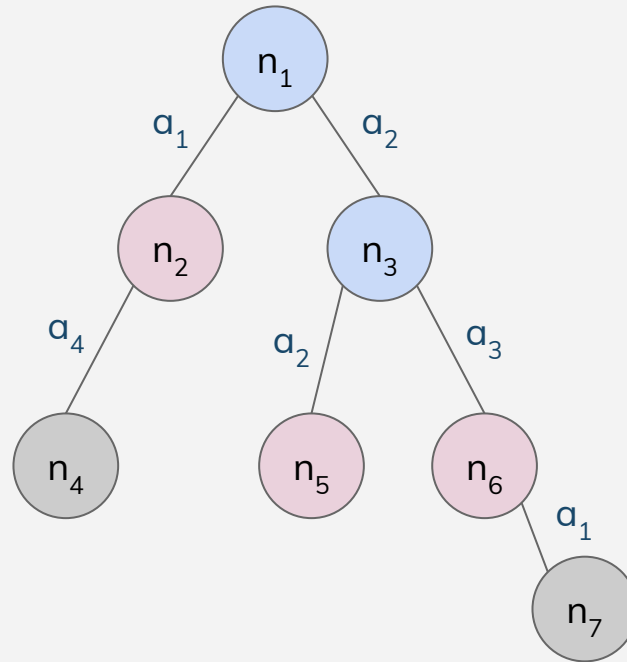


Breadth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6\}$

$Fr = \{n_2, n_5, n_6\}$

$Exp = \{n_1, n_3\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



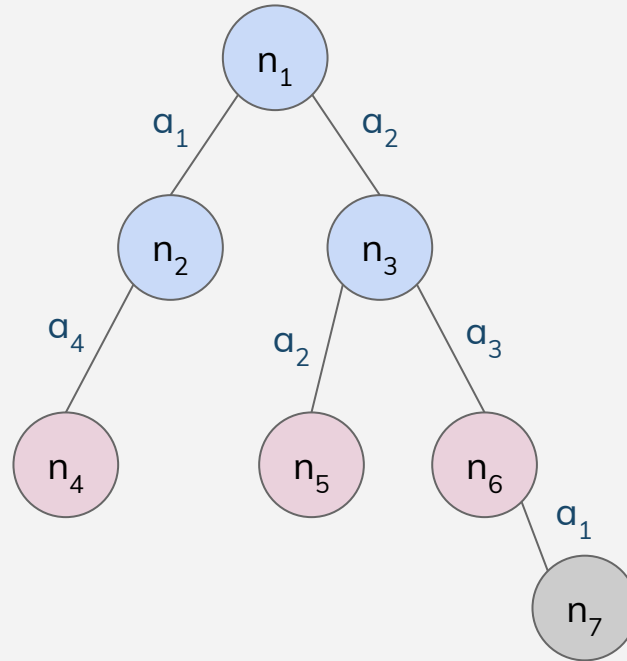


Breadth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4\}$

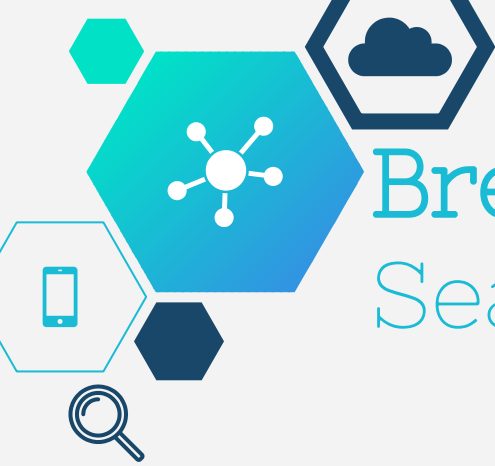
$Fr = \{n_5, n_6, n_4\}$

$Exp = \{n_1, n_3, n_2\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



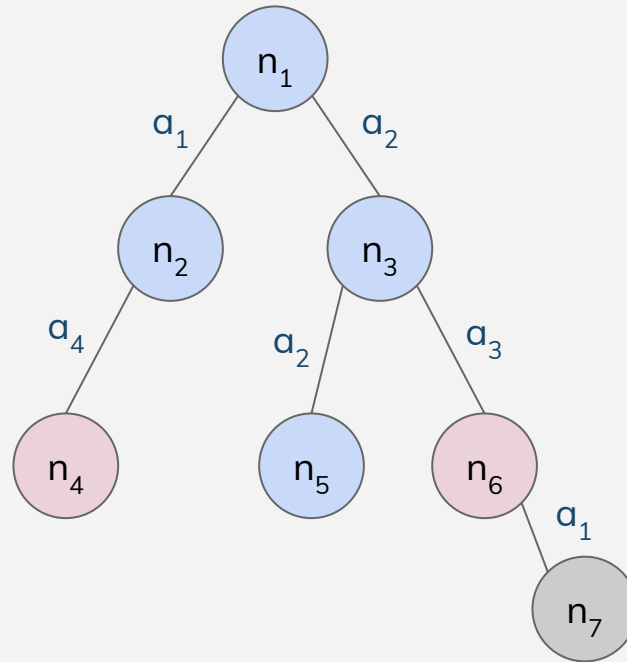


Breadth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4\}$

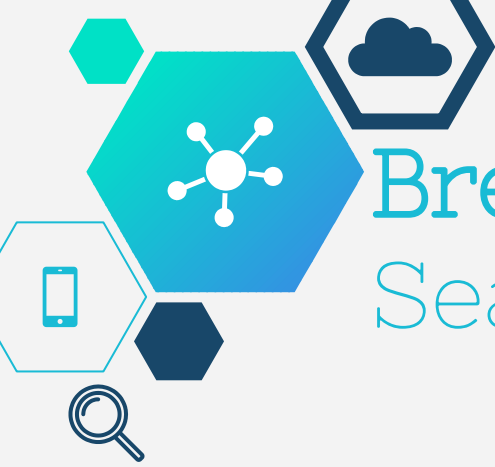
$Fr = \{n_6, n_4\}$

$Exp = \{n_1, n_3, n_2, n_5\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



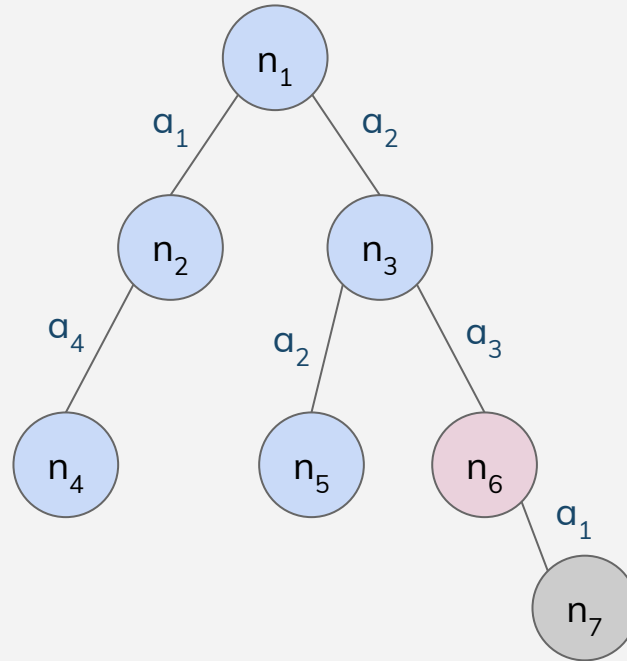


Breadth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4\}$

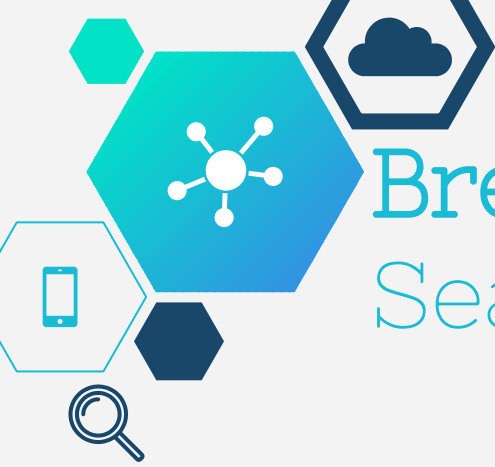
$Fr = \{n_6\}$

$Exp = \{n_1, n_3, n_2, n_5, n_4\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



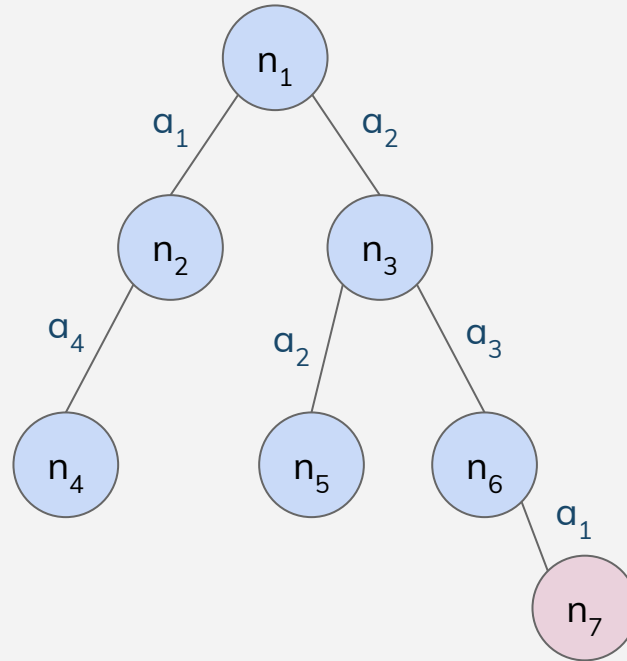


Breadth First Search

$Tr = \{ n_1, n_2, n_3, n_5, n_6, n_4, n_7 \}$

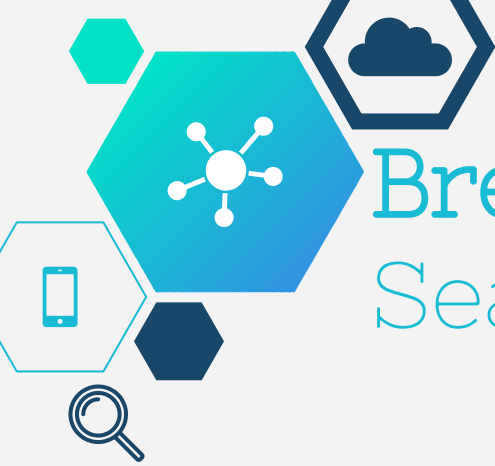
$Fr = \{ n_7 \}$

$Exp = \{ n_1, n_3, n_2, n_5, n_4, n_6 \}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



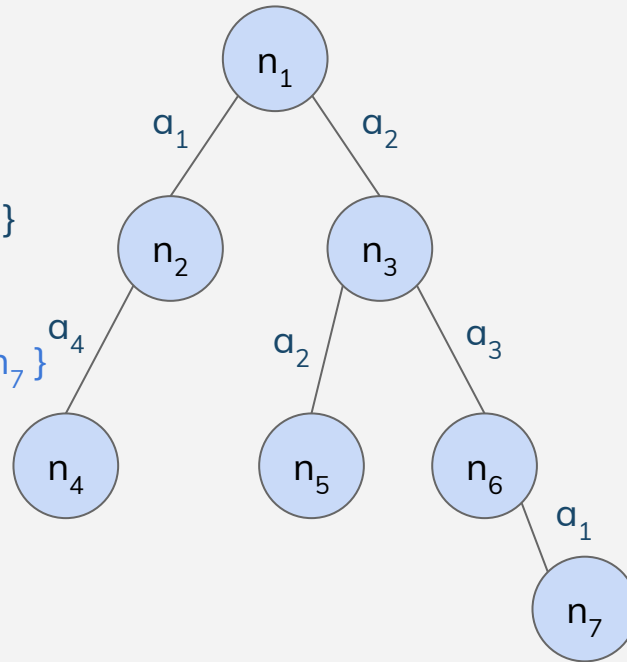


Breadth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4, n_7\}$

$Fr = \{\}$

$Exp = \{n_1, n_3, n_2, n_5, n_4, n_6, n_7\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados





Uniform Cost Search

- Expande el nodo con menor costo acumulado.
- Idéntico a BFS en problemas de costo uniforme.
- **Óptima** si sucede alguna de las siguientes:
 - $g(\text{sucesor}(n)) \geq g(n) ; \forall n$

Dicho de otra forma..

$g(a) \geq 0 ; \forall a: \text{acción}$

(si el costo de una acción es < 0 , no es óptima)



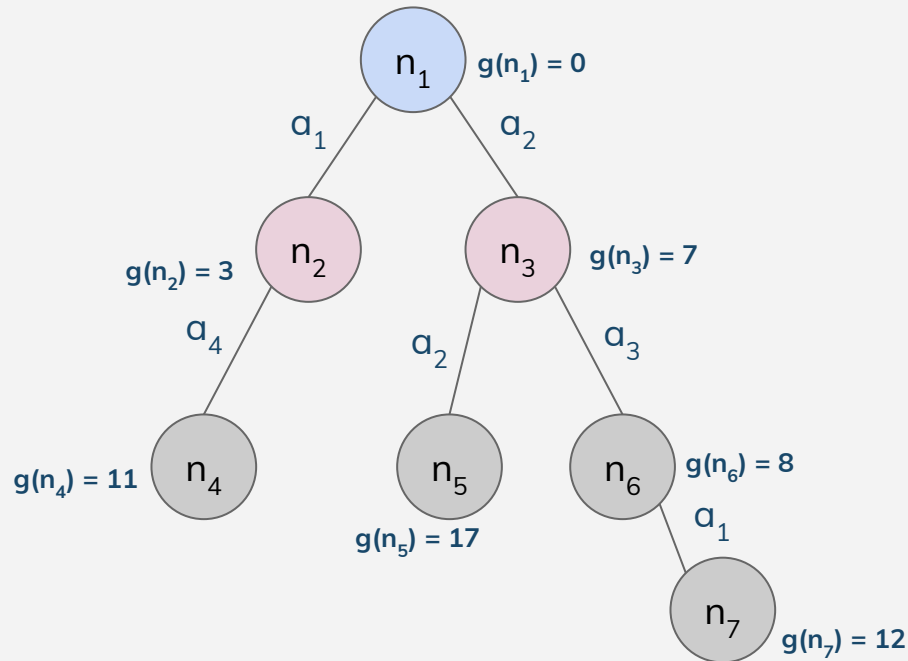


Uniform Cost Search

$Tr = \{n_1, n_2, n_3\}$

$Fr = \{n_2, n_3\}$

$Exp = \{n_1\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



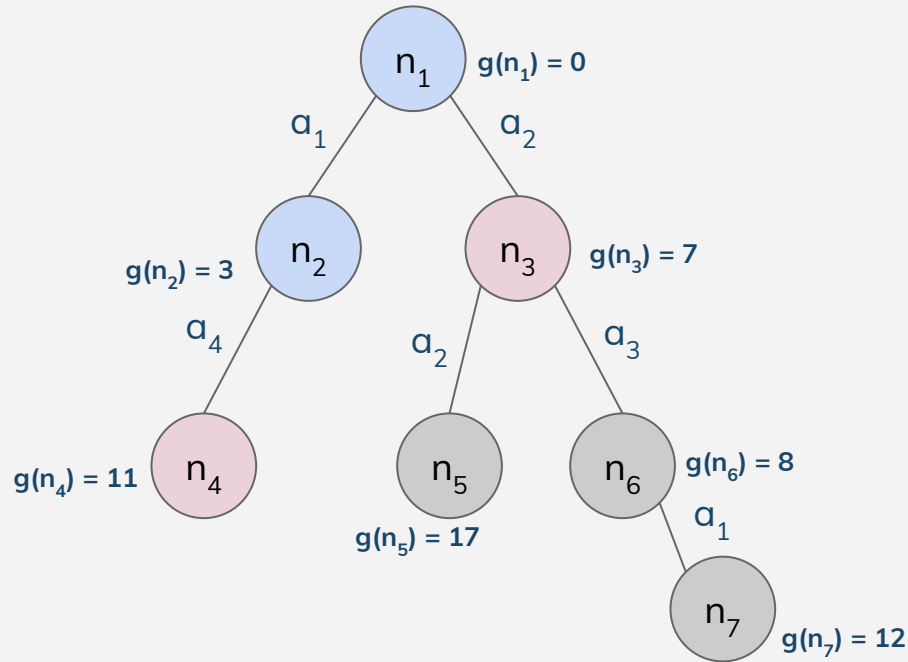


Uniform Cost Search

$Tr = \{n_1, n_2, n_3, n_4\}$

$Fr = \{n_3, n_4\}$

$Exp = \{n_1, n_2\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



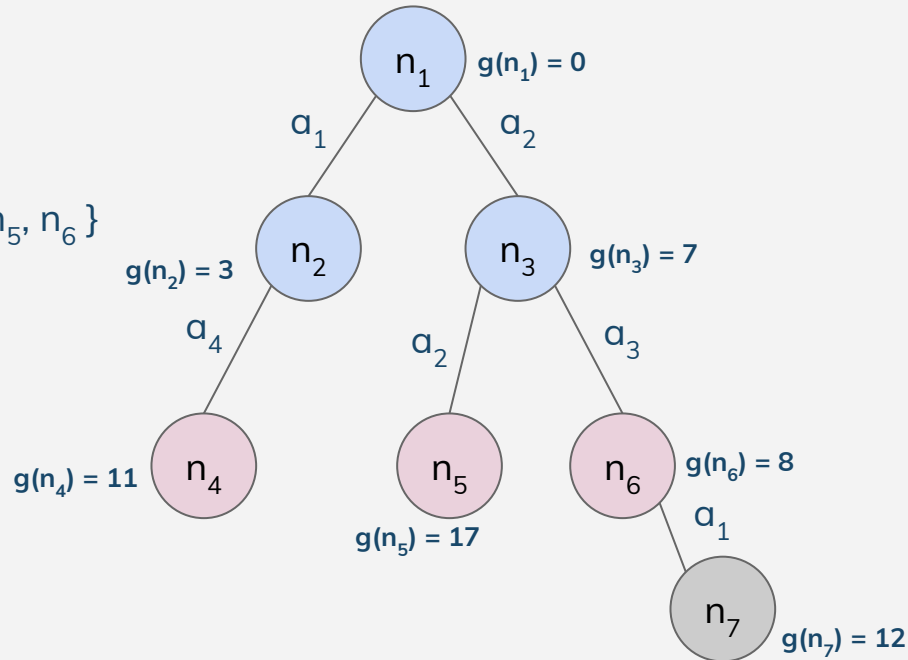


Uniform Cost Search

$Tr = \{n_1, n_2, n_3, n_4, n_5, n_6\}$

$Fr = \{n_4, n_5, n_6\}$

$Exp = \{n_1, n_2, n_3\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



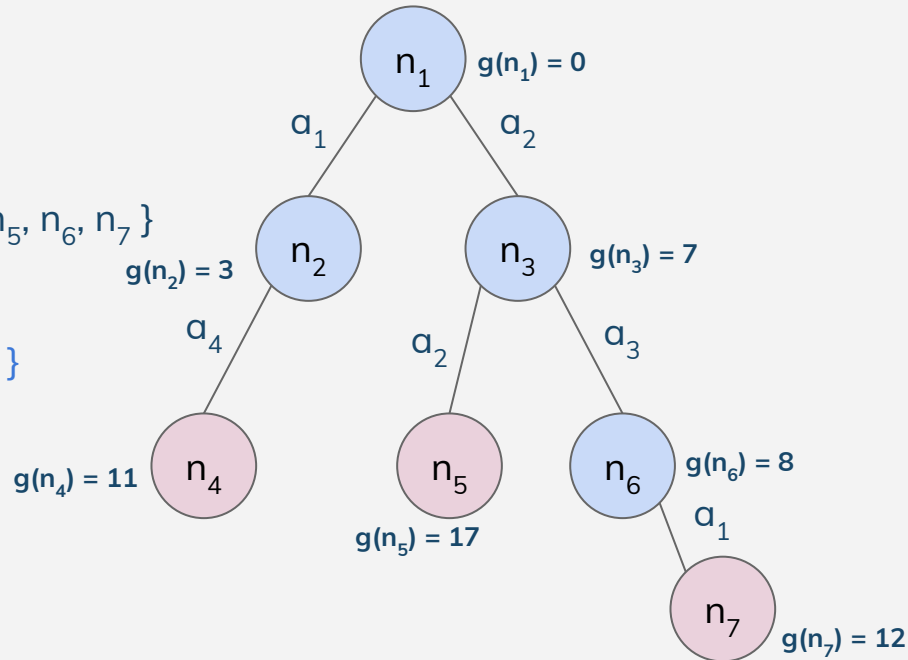


Uniform Cost Search

$Tr = \{n_1, n_2, n_3, n_4, n_5, n_6, n_7\}$

$Fr = \{n_4, n_5, n_7\}$

$Exp = \{n_1, n_2, n_3, n_6\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



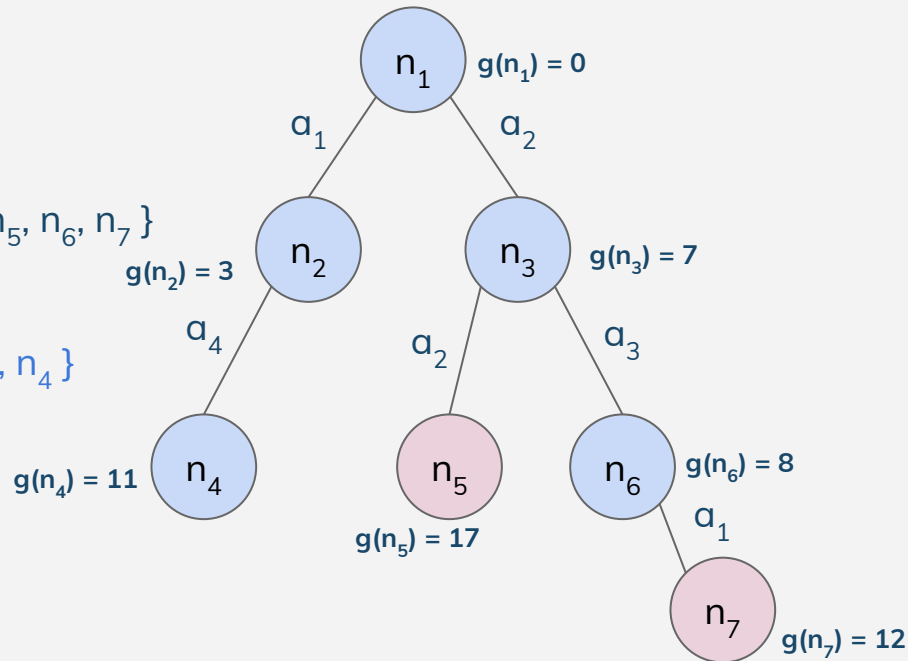


Uniform Cost Search

$Tr = \{n_1, n_2, n_3, n_4, n_5, n_6, n_7\}$

$Fr = \{n_5, n_7\}$

$Exp = \{n_1, n_2, n_3, n_6, n_4\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados





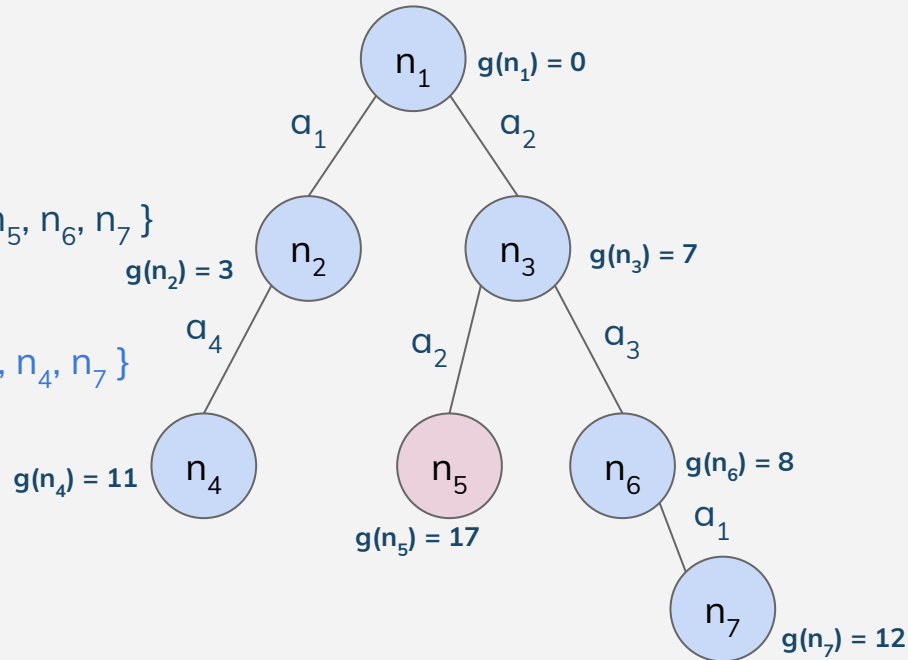
Uniform Cost Search

-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados

$Tr = \{n_1, n_2, n_3, n_4, n_5, n_6, n_7\}$

$Fr = \{n_5\}$

$Exp = \{n_1, n_2, n_3, n_6, n_4, n_7\}$





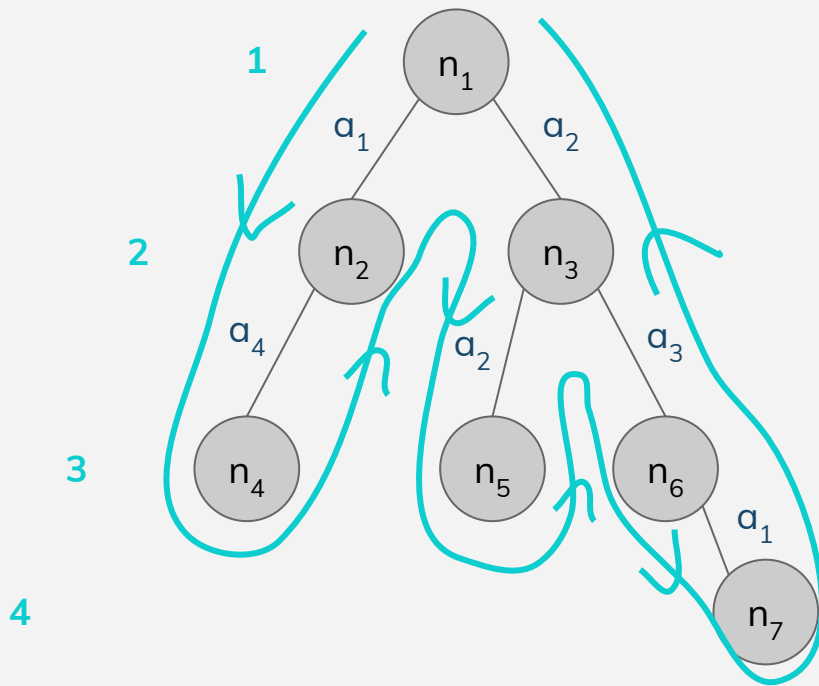
Depth First Search

- Expande el nodo de mayor profundidad.
- Suele ser de menor complejidad espacial, sobre todo en su version sin estados repetidos (más adelante).
- Suele ser más rápido que BFS cuando existen muchas soluciones.
- No es óptima.





Depth First Search



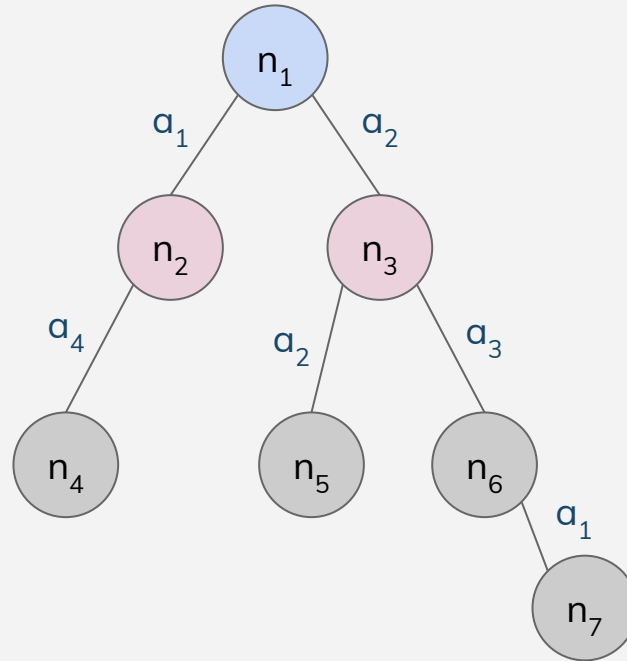


Depth First Search

$Tr = \{n_1, n_2, n_3\}$

$Fr = \{n_2, n_3\}$

$Exp = \{n_1\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



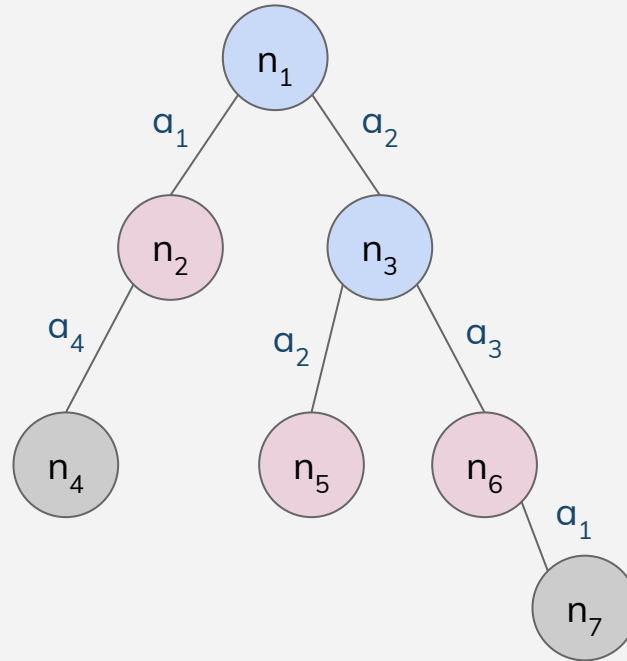


Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6\}$

$Fr = \{n_2, n_5, n_6\}$

$Exp = \{n_1, n_3\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



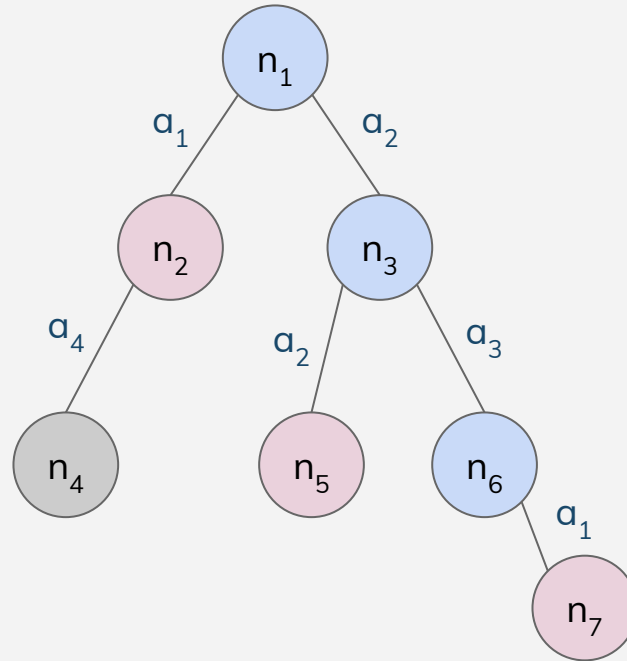


Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_7\}$

$Fr = \{n_2, n_5, n_7\}$

$Exp = \{n_1, n_3, n_6\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



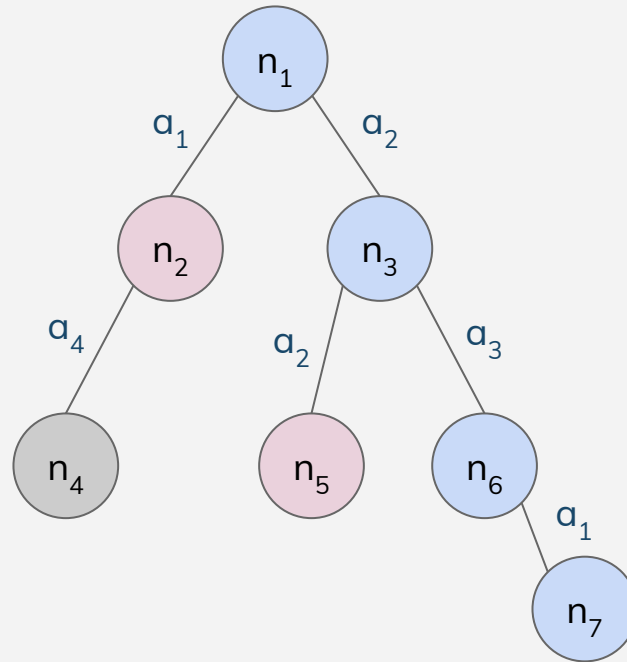


Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_7\}$

$Fr = \{n_2, n_5\}$

$Exp = \{n_1, n_3, n_6, n_7\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



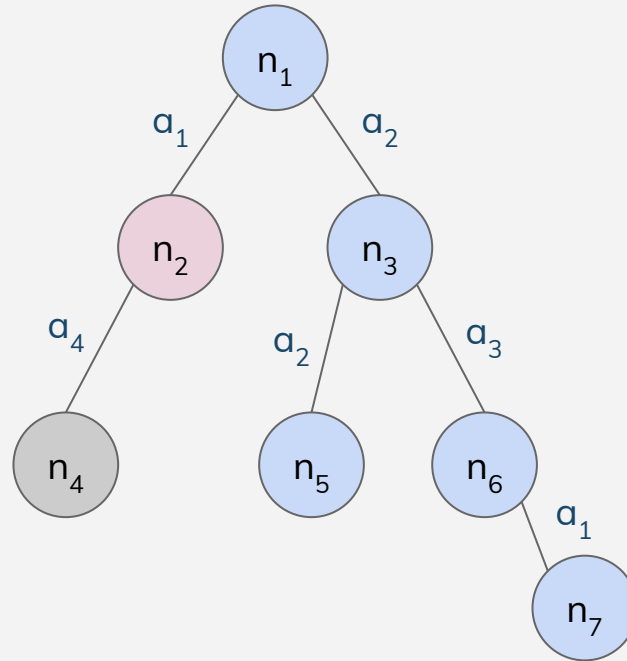


Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_7\}$

$Fr = \{n_2\}$

$Exp = \{n_1, n_3, n_6, n_7, n_5\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados





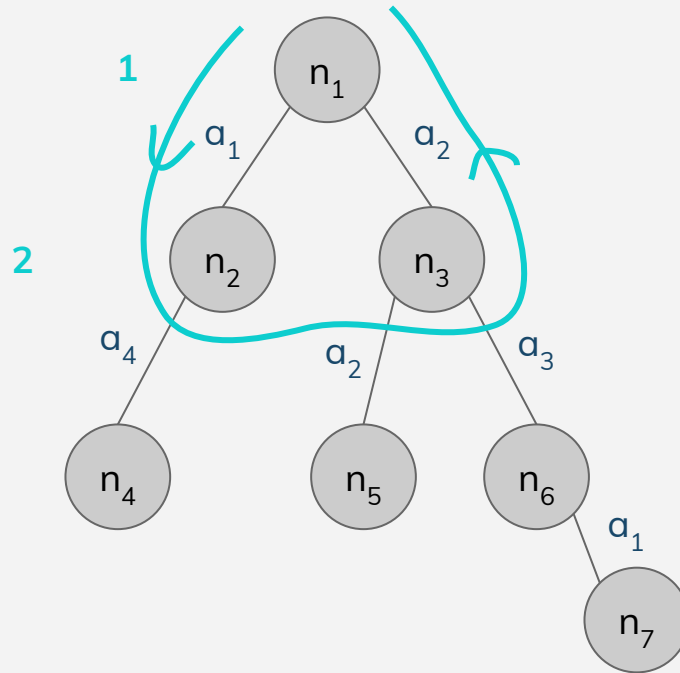
Depth Limited Search

- Idem DFS sólo que se limita la profundidad.
- No es completa ni óptima.





Depth Limited Search



Por ejemplo,
solo llego a
profundidad 2



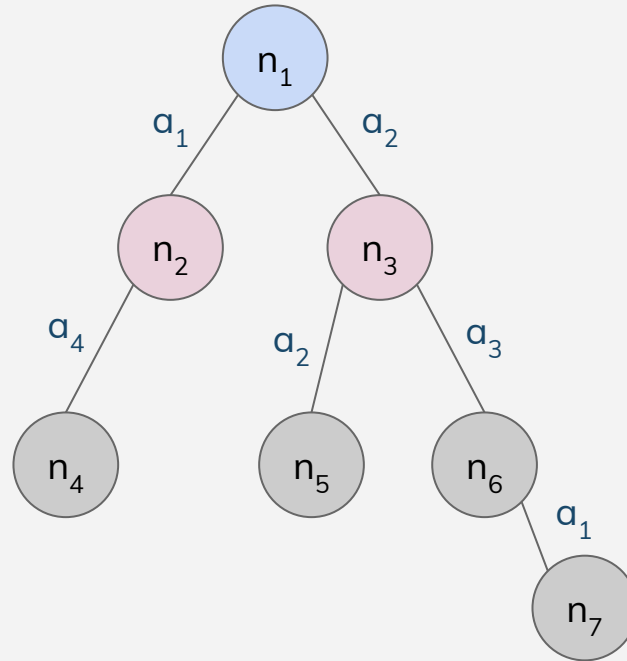


Depth Limited Search

$Tr = \{n_1, n_2, n_3\}$

$Fr = \{n_2, n_3\}$

$Exp = \{n_1\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



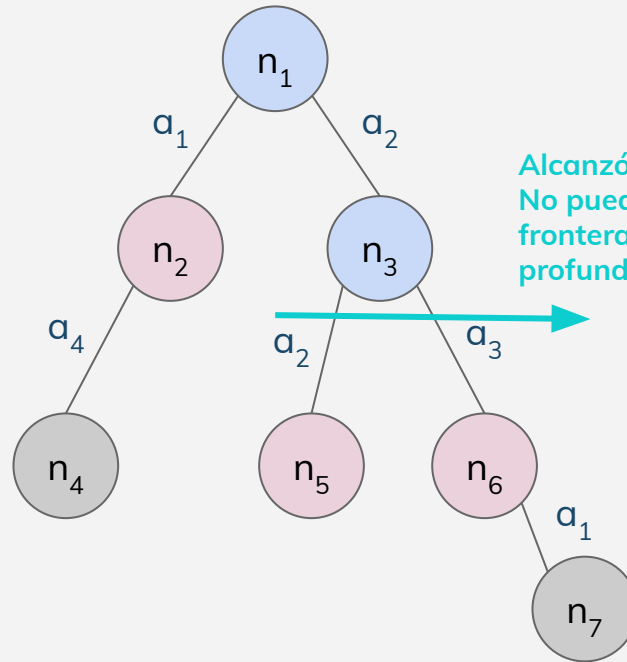


Depth Limited Search

$Tr = \{n_1, n_2, n_3, n_5, n_6\}$

$Fr = \{n_2, n_5, n_6\}$

$Exp = \{n_1, n_3\}$



Alcanzó el límite.
No puede tomar nodos
frontera de mayor
profundidad.

-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



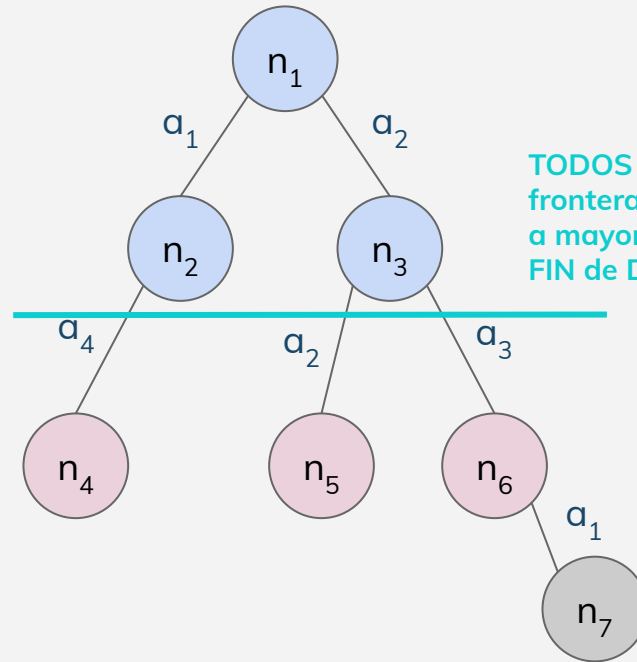


Depth Limited Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4\}$

$Fr = \{n_5, n_6, n_4\}$

$Exp = \{n_1, n_3, n_2\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados

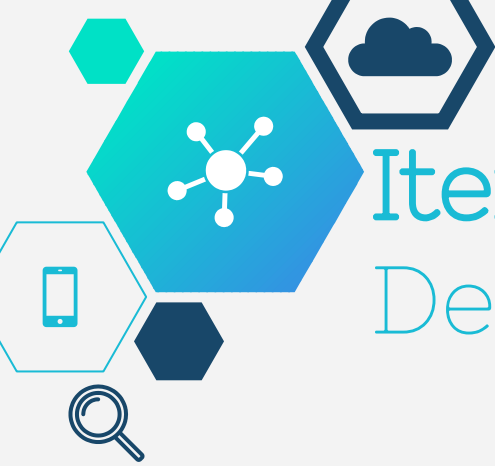




Iterative Deepening Depth First Search

- DLS donde la profundidad varía según cierto criterio.
- Se puede implementar manteniendo los nodos hoja que no fueron expandidos por cota de profundidad entre cada iteración.
- Por ejemplo, podría configurarse con un profundidad inicial de 1 y creciente en 1 en cada iteración - sería un BFS (más costoso si no mantiene el árbol al reiniciar).
- Con un valor infinito, es un DFS.

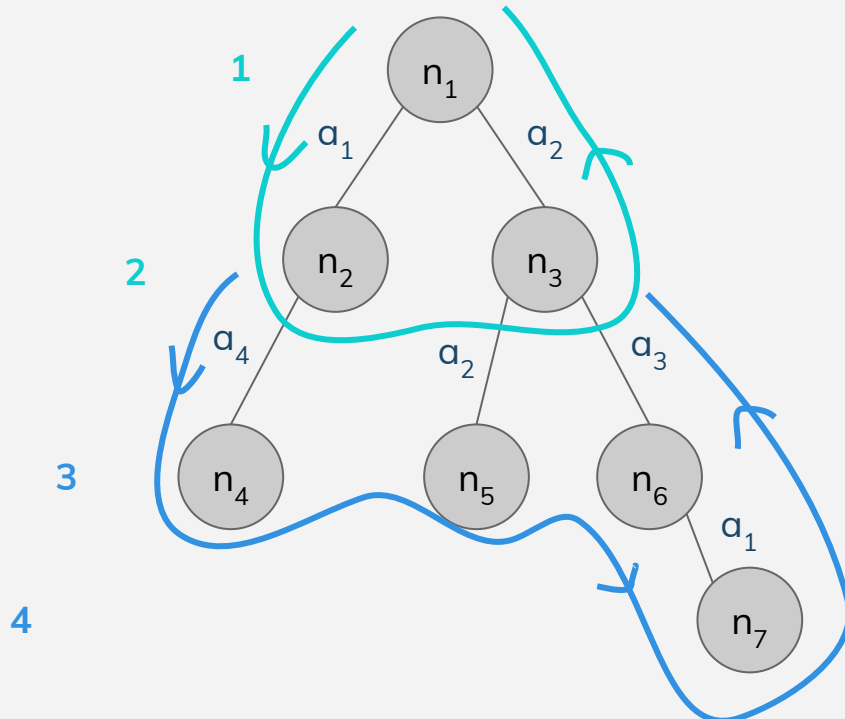


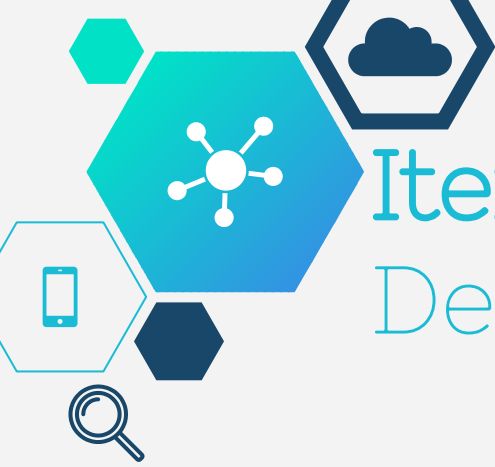


Iterative Deepening Depth First Search

Primero llego a
profundidad 2.

Luego, llego a
profundidad 4.



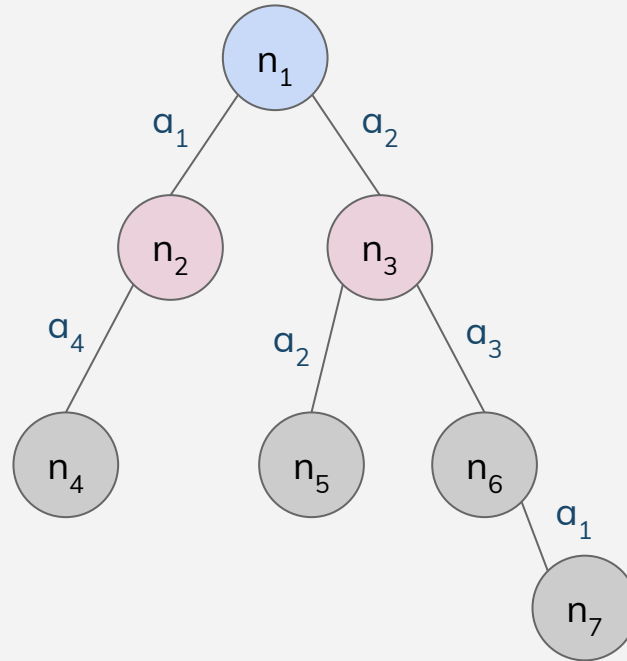


Iterative Deepening Depth First Search

$Tr = \{n_1, n_2, n_3\}$

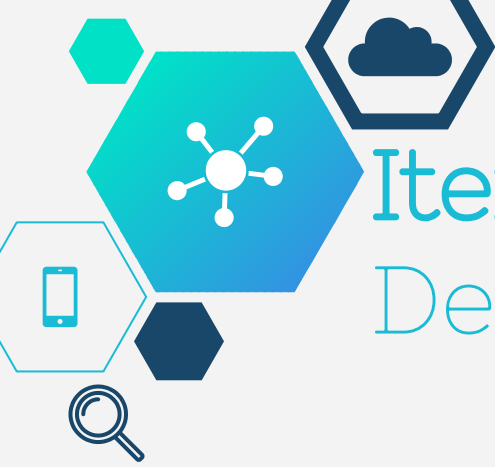
$Fr = \{n_2, n_3\}$

$Exp = \{n_1\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



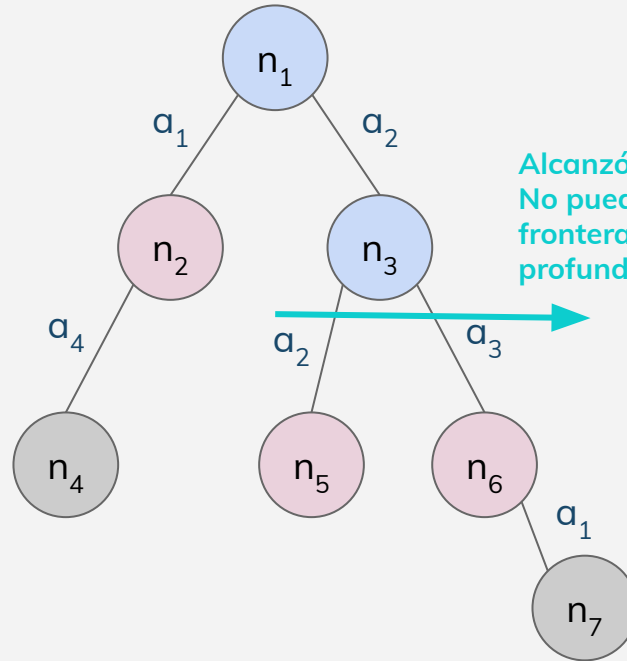


Iterative Deepening Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6\}$

$Fr = \{n_2, n_5, n_6\}$

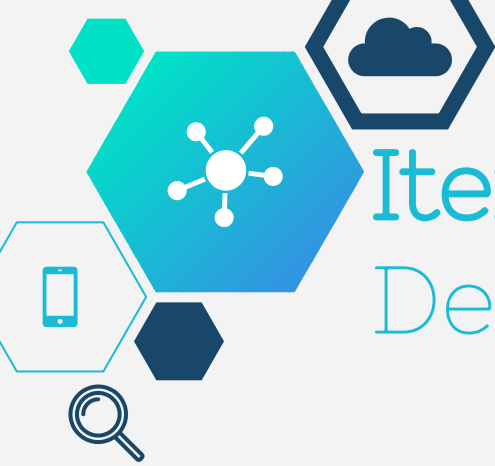
$Exp = \{n_1, n_3\}$



Alcanzó el límite.
No puede tomar nodos
frontera de mayor
profundidad.

-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados





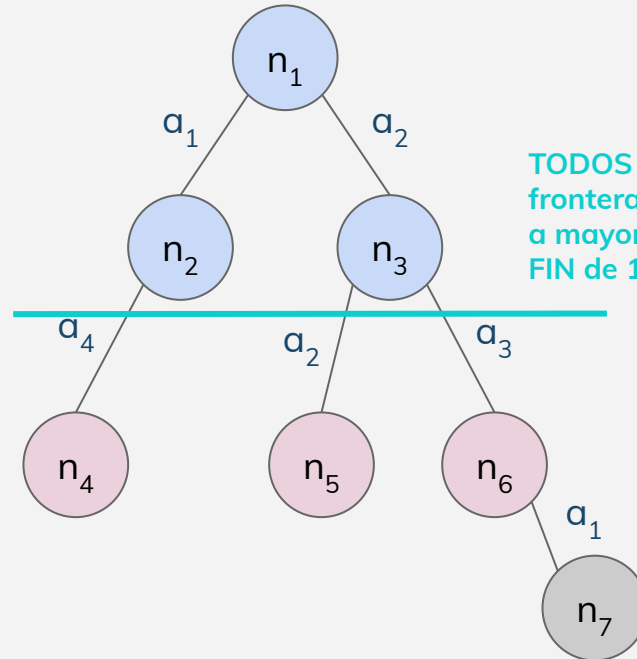
Iterative Deepening Depth First Search

-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4\}$

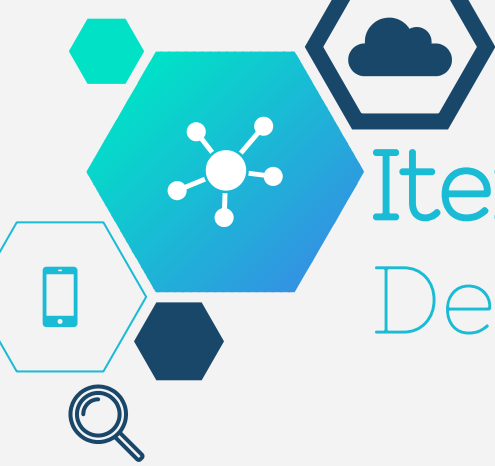
$Fr = \{n_5, n_6, n_4\}$

$Exp = \{n_1, n_3, n_2\}$



TODOS los nodos
frontera se encuentran
a mayor profundidad.
FIN de 1ra iteración.





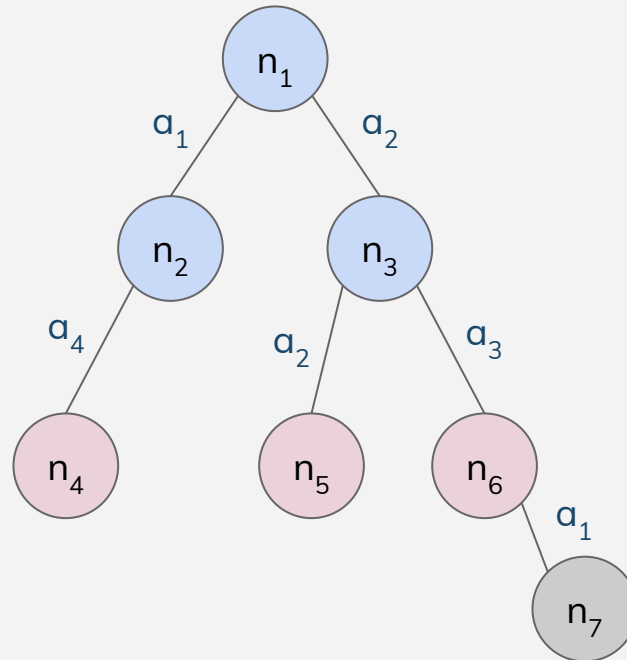
Iterative Deepening Depth First Search

-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4\}$

$Fr = \{n_5, n_6, n_4\}$

$Exp = \{n_1, n_3, n_2\}$



Límite de la segunda
iteración





Iterative Deepening Depth First Search

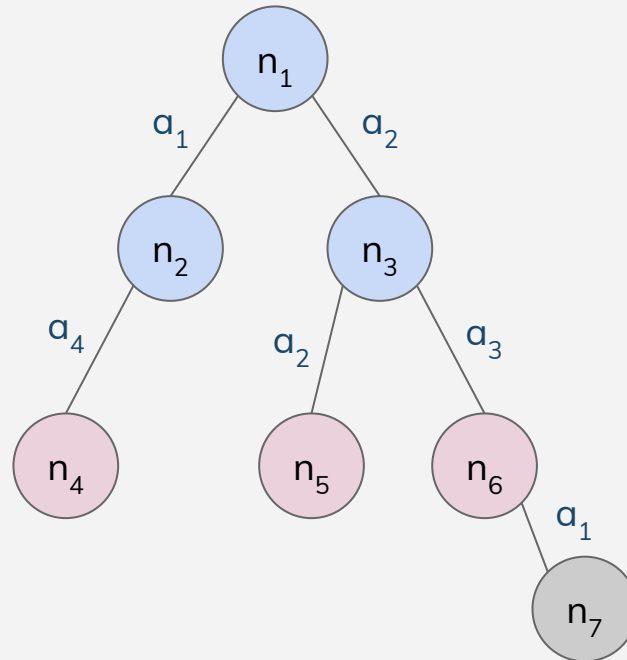
-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4\}$

$Fr = \{n_5, n_6, n_4\}$

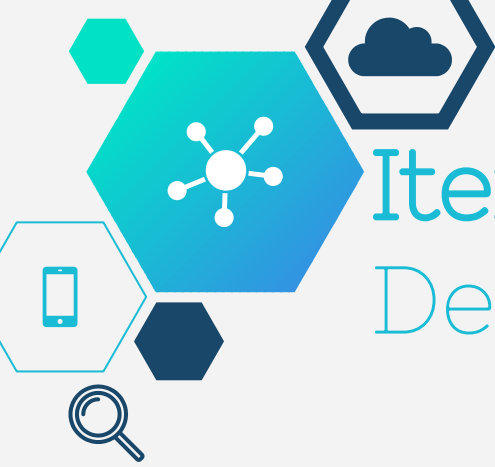
$Exp = \{n_1, n_3, n_2\}$

A partir de la frontera
de la iteración anterior.



Límite de la segunda
iteración



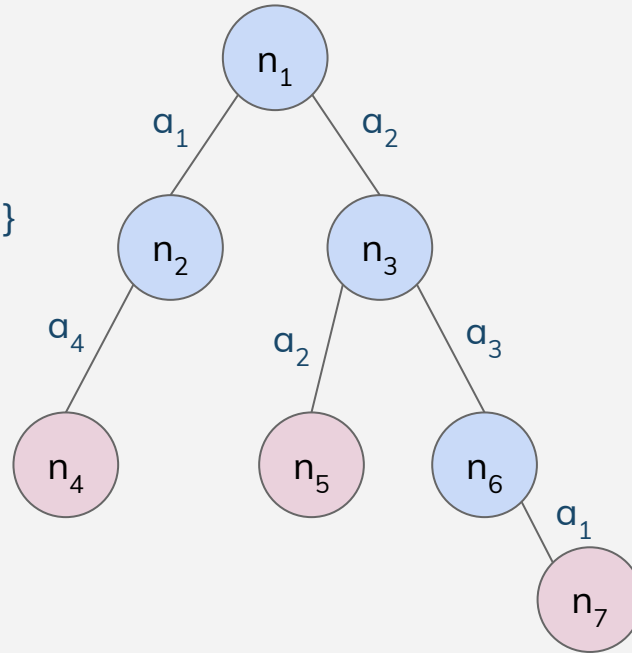


Iterative Deepening Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4, n_7\}$

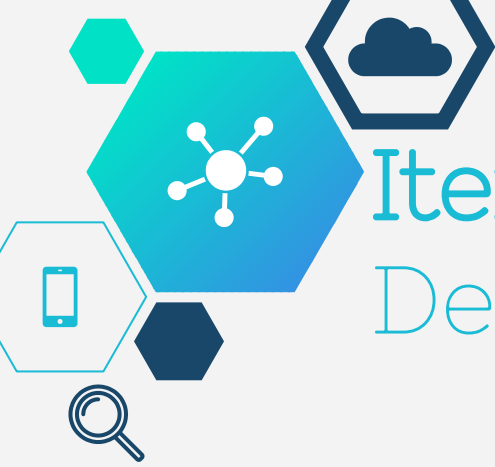
$Fr = \{n_5, n_4, n_7\}$

$Exp = \{n_1, n_3, n_2, n_6\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



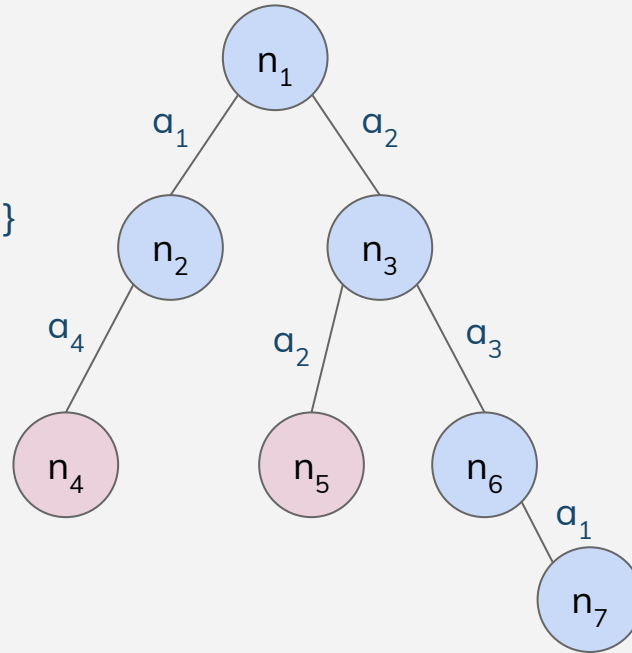


Iterative Deepening Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4, n_7\}$

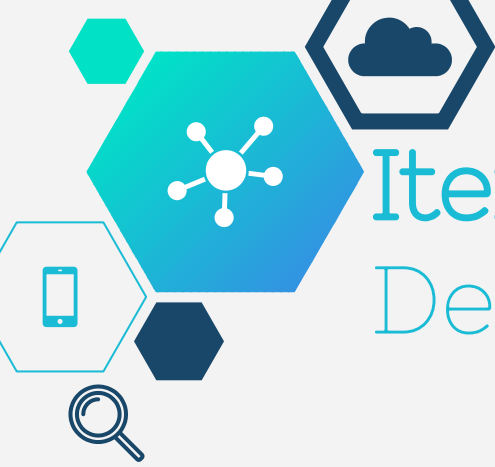
$Fr = \{n_5, n_4\}$

$Exp = \{n_1, n_3, n_2, n_6, n_7\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



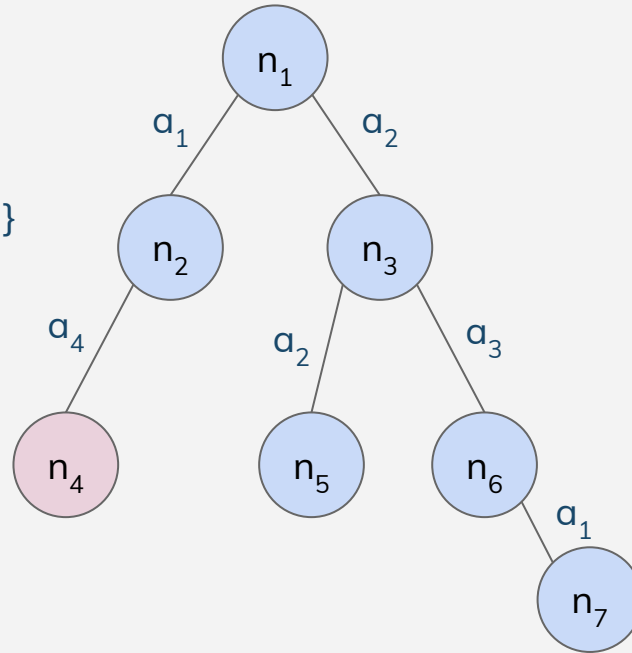


Iterative Deepening Depth First Search

$Tr = \{n_1, n_2, n_3, n_5, n_6, n_4, n_7\}$

$Fr = \{n_4\}$

$Exp = \{n_1, n_3, n_2, n_6, n_7, n_5\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados





Estados Repetidos

Motivación:

- No regresar a estados ancestros.
- No crear rutas que tengan ciclos.
- No volver a expandir estados ya expandidos previamente
 - Muy útil para DFS y BFS
 - Ojo con DLS e IDDFS !!
 - Si existe una solución a profundidad N , podemos no encontrarla.
- Compromiso vs el costo de almacenar y chequear estados.



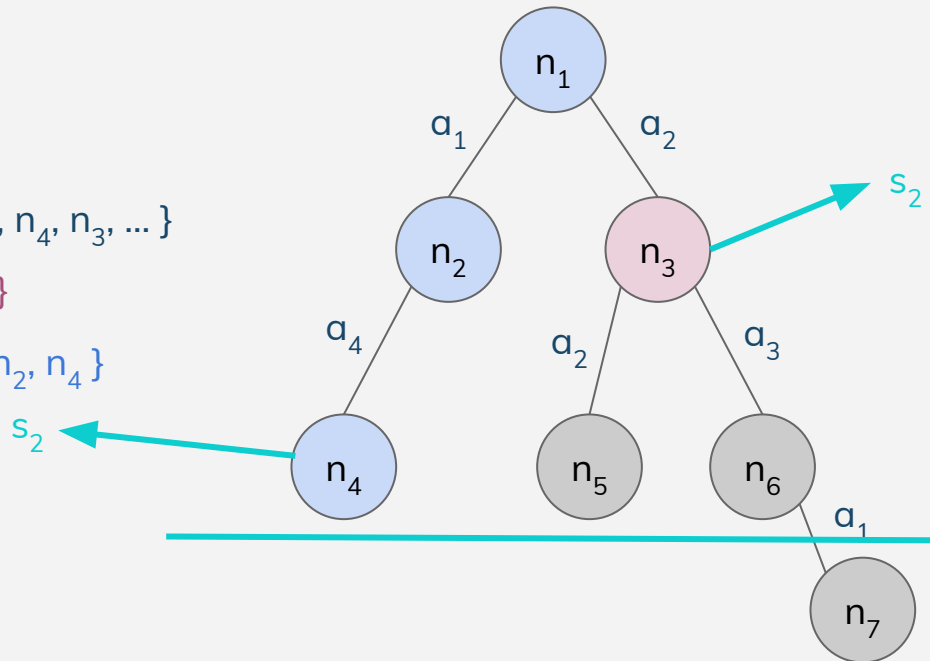


Estados Repetidos

$Tr = \{ n_1, n_2, n_4, n_3, \dots \}$

$Fr = \{ n_3, \dots \}$

$Exp = \{ n_1, n_2, n_4 \}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



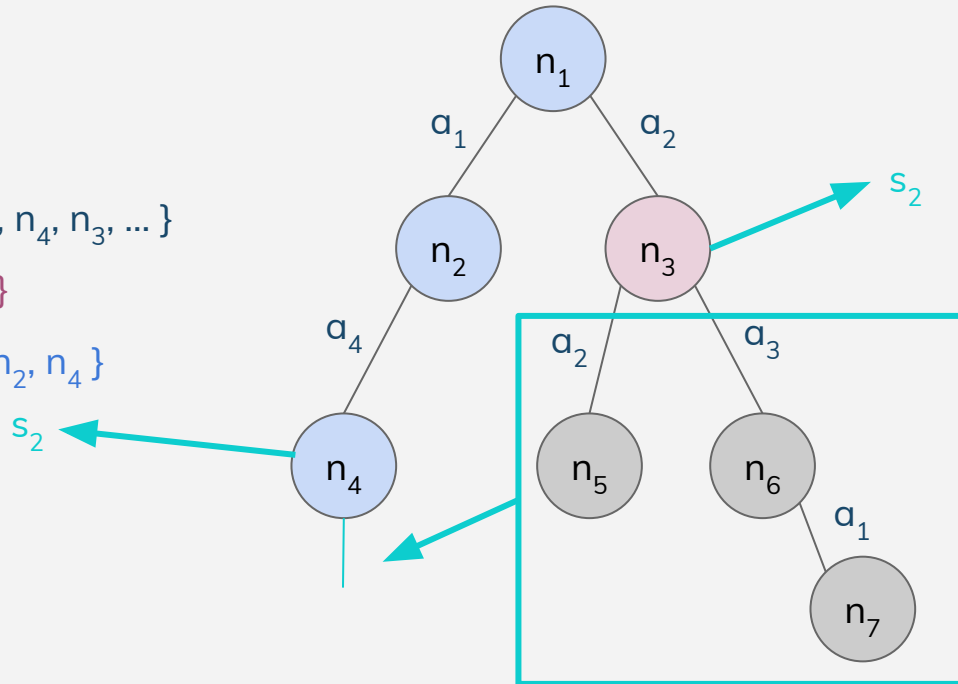


Estados Repetidos

$Tr = \{n_1, n_2, n_4, n_3, \dots\}$

$Fr = \{n_3, \dots\}$

$Exp = \{n_1, n_2, n_4\}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados



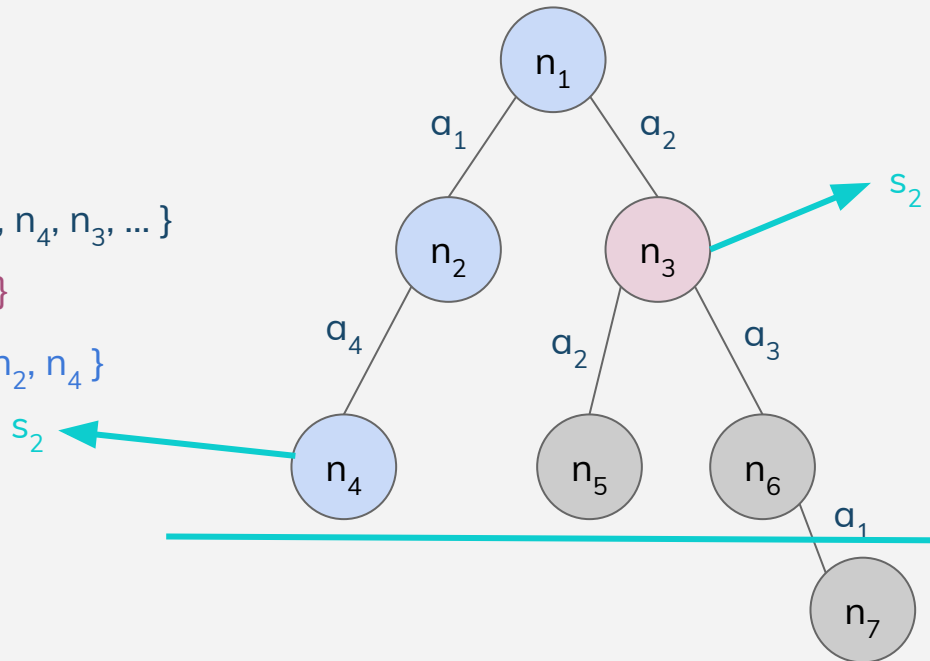


Estados Repetidos

$Tr = \{ n_1, n_2, n_4, n_3, \dots \}$

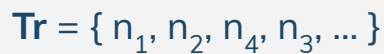
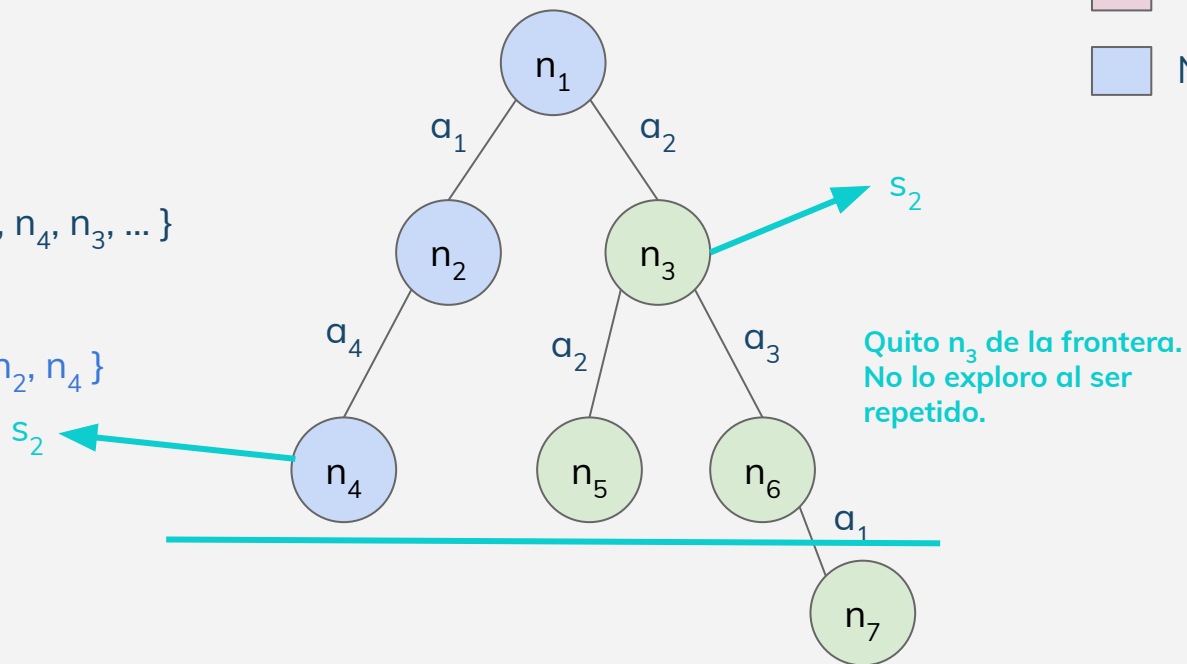
$Fr = \{ n_3, \dots \}$

$Exp = \{ n_1, n_2, n_4 \}$



-  Nodos no explorados
-  Nodos frontera
-  Nodos explorados




$$\text{Exp} = \{n_1, n_2, n_4\}$$




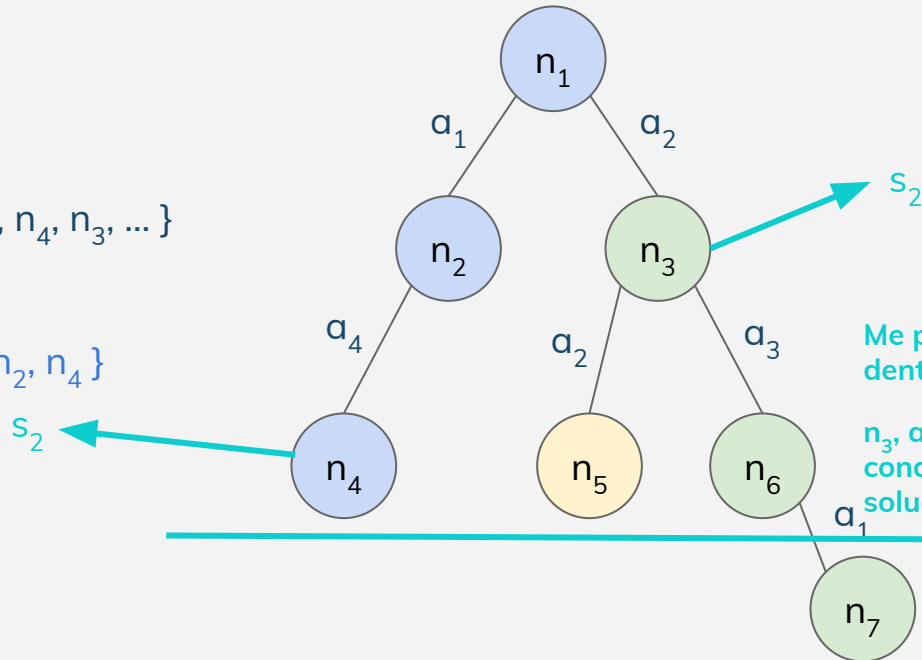
Estados Repetidos

- Goal
- Nodos ignorados
- Nodos no explorados
- Nodos frontera
- Nodos explorados

$Tr = \{ n_1, n_2, n_4, n_3, \dots \}$

$Fr = \{ \dots \}$

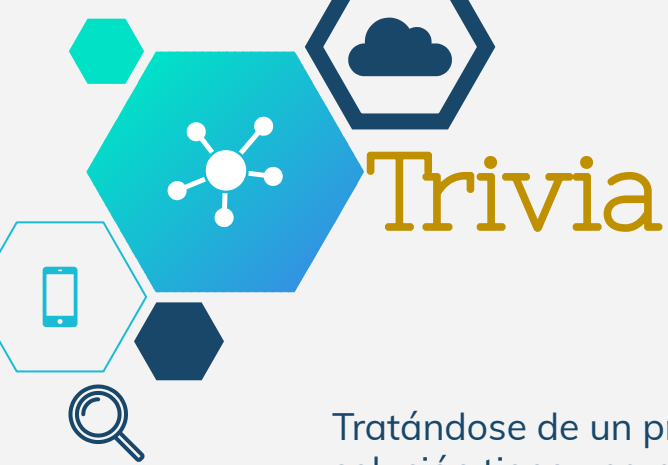
$Exp = \{ n_1, n_2, n_4 \}$



Me perdí n_5 que es goal dentro del límite actual.

n_3 , además, me conducía a una solución óptima.





Trivia

Tratándose de un problema de costo uniforme, se sabe que la solución tiene una profundidad < 1000 . El problema tiene un factor de ramificación de 10.

Se quiere hallar una solución **óptima**. Unos colegas hacen unos cálculos y encuentran que el hardware no cuenta con la memoria suficiente para correr BFS.

¿ Qué solución propondrían al equipo ?



Por supuesto, omitir la solución trivial
"mejorar el hardware".





Práctica

Para el problema dado, definir las siguientes características:

1. Arquitectura de estado
2. Acciones
3. Estado Inicial
4. Espacio de estados
5. Condición de terminación

- 8 reinas
- 8 Puzzle
- Misioneros y Caníbales
- Sudoku (1-4)
- Laberinto
- Travelling Salesman

